CloudXR Viewer

with Foveated Streaming

A Literate Program

pfiguero@uniandes.edu.co

November 4, 2026

Contents

1 Introduction **1**
- 1.1 Why foveation . . . 1
- 1.2 Two backends . . . 1
- 1.3 How to read this document . . . 2
- 1.4 Plan of the book . . . 2
- 1.5 License headers . . . 3

2 The Application and Its Scenes **4**
- 2.1 The application . . . 4
  - 2.1.1 Scenes on visionOS . . . 5
  - 2.1.2 Two immersive spaces . . . 7
  - 2.1.3 The scene on iOS . . . 8
  - 2.1.4 Disconnection alerts . . . 9
- 2.2 The model . . . 10
- 2.3 The application delegate . . . 12
- 2.4 Test hook . . . 13

3 A Vocabulary Shared by Both Backends **14**
- 3.1 Backends . . . 14
- 3.2 Status . . . 15
- 3.3 Channels . . . 16
- 3.4 The session protocol . . . 16

4 The CloudXRKit Backend **17**
- 4.1 The adapter . . . 17
- 4.2 The configuration form . . . 19
- 4.3 Connecting with CloudXRKit . . . 22
- 4.4 Guest mode . . . 24

5 The Foveated Streaming Backend **26**
- 5.1 Endpoints . . . 26
- 5.2 The session . . . 27
- 5.3 Connecting . . . 29
- 5.4 Build configuration . . . 30
- 5.5 Entitlements and property lists . . . 31

6 The visionOS User Interface **33**
- 6.1 The window manager . . . 33
- 6.2 Following the connection . . . 35
- 6.3 Following the immersive space . . . 36
- 6.4 The launch window . . . 37
- 6.5 The configuration form . . . 38

6.6 The content window 41
6.7 Tabs of the content window 44
6.8 The immersive views 47
6.9 Styles and placement helpers 48

7 The iOS User Interface **50**
7.1 The launch view 50
7.2 The configuration form 51
7.3 The streaming view 54
7.4 Overlay buttons 56

8 Opaque Data Channels **58**
8.1 Sending 58
8.2 The panel 59

9 Remaining Pieces **62**
9.1 Form helpers 62
9.2 Feedback 64
9.3 Styles and constants 65
9.4 Debug printing 67

Bibliography **69**

Index of Chunks **70**

Chapter 1

Introduction

This book retells the source code of a client application for Apple Vision Pro and iPhone that shows immersive content rendered on another computer. The application is a fork of NVIDIA's *CloudXR Generic Viewer* [3], the reference client for CloudXR on Apple platforms. The fork keeps everything the original does and adds a second way of streaming: Apple's *Foveated Streaming* framework [5], which visionOS provides as part of the system, and its iOS counterpart, the *StreamingSession* package [6].

1.1 Why foveation

The human eye sees fine detail only in a small region around the point of fixation, the fovea, which spans a couple of degrees of the visual field. Acuity falls quickly with eccentricity: a few degrees away from the fixation point the eye already resolves several times less detail. *Foveated rendering* exploits this by spending effort where the user looks and saving it in the periphery; Guenter et al. [7] showed that rendering three nested layers at decreasing resolution around a tracked gaze point is indistinguishable from full resolution when the eye tracker is fast enough.

In remote rendering the scarce resource is not only GPU time but the network: a stereo stream at the resolution of Vision Pro's displays needs many tens of megabits per second. With Foveated Streaming the headset reports the *approximate region* the user is looking at to the CloudXR Runtime on the server, which uses it to distribute the video encoder's bit rate: the gaze region is encoded at full quality and the periphery more coarsely. The foveation therefore happens in the encoder, driven by the operating system, and not in this application. Two consequences shape the code:

- The client never sees gaze data. visionOS sends the focus region to the CloudXR Runtime inside the session, and the runtime does not hand it to the streamed OpenXR application either (NVIDIA's documentation states this explicitly [4]). Eye-tracking privacy is kept by construction.

- The client does not draw the stream itself. The system presents it in an immersive space that the application declares with `ImmersiveSpace(foveatedStreaming:)`, and the session opens and closes that space on its own.

1.2 Two backends

The configuration screen lets the user pick the *streaming backend*:

Backend	Platform	Library
CloudXR	visionOS, iOS	CloudXRKit Swift package (the original code path)
Foveated Streaming	visionOS	FoveatedStreaming system framework
Streaming Session	iOS	StreamingSession package, same API without eye tracking

Having both in one application makes it possible to connect to the same server with and without foveation and compare. It also keeps the fork close to NVIDIA's code, which matters because the work is meant to be offered back to the original repository: the application keeps its name, its targets and its hand-maintained Xcode project, and the changes are as small as the design allows.

The two libraries have similar but not identical APIs. Where CloudXRKit has a `Session` with a `state`, Foveated Streaming has a `FoveatedStreamingSession` with a `status`; CloudXRKit disconnects synchronously, Foveated Streaming asynchronously; CloudXRKit's message channels return a `Bool` when sending, Foveated Streaming's throw. Rather than sprinkle the views with `if backend == ...` tests, the fork introduces a small protocol, `ViewerSession` (Chapter 3), and one adapter per library (Chapters 4 and 5). The views talk to the protocol; only the places where the libraries truly differ (how a connection is configured, how the stream is drawn) look at the backend.

1.3 How to read this document

This is a *literate program* in the sense of Donald Knuth [1]: a document written for people, from which the program for the machine is extracted. The code appears in *chunks*. Each chunk has a name and a number, and is written

$$\langle \textit{name}\ \texttt{n} \rangle \equiv$$

where it is defined. A chunk may use other chunks by name. When the same name is defined again, the new text is appended to the old, and the header reads $+\equiv$. Below each chunk, a note tells where it is continued and where it is used. The chunk index at the end of the book lists every chunk.

A chunk named like a file is a *root*: *tangling* follows its references and writes the file. The roots of this web are exactly the Swift sources of the application, its build configuration, its entitlements and its two `Info.plist` files, so tangling rebuilds them from this document. *Weaving* produces the LaTeX you are reading. The directory `LiterateP` holds the web (`chapters/*.nw`), the tool `lit.py` that does both jobs using Norman Ramsey's noweb notation [2], and a `Makefile`. Running `make check` tangles the web and compares every file, byte for byte, with the sources in the repository, so this document cannot drift from the program without the check noticing. The Xcode project file is left out: Xcode maintains it.

Many of NVIDIA's files end without a final newline. Their last chunk ends with the marker `@nonl`, which `lit.py` understands, and the note under the chunk says so.

1.4 Plan of the book

Chapter 2 presents the application object, its scenes and the shared model. Chapter 3 defines the backend-neutral vocabulary: backends, statuses, channels and the `ViewerSession` protocol. Chapter 4 adapts CloudXRKit to it and shows how a classic connection is configured; Chapter 5 does the same for Foveated Streaming and covers the build settings, entitlements and property lists it needs. Chapter 6 is the visionOS user interface and the window manager that coordinates windows and immersive spaces; Chapter 7 the iOS one. Chapter 8 is the opaque data channel panel, and Chapter 9 collects the remaining helpers.

1.5 License headers

Every Swift file of NVIDIA's code starts with the MIT permission notice followed by NVIDIA's copyright line. Files written for the fork carry the same notice with their own copyright line. The notice is written once here.

The permission notice, eighteen lines of comments ending with an empty comment line:

1 ⟨*MIT permission notice* 1⟩ ≡

```
// Permission is hereby granted, free of charge, to any person obtaining a copy
// of this software and associated documentation files (the "Software"), to deal
// in the Software without restriction, including without limitation the rights
// to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
// copies of the Software, and to permit persons to whom the Software is furnished
// to do so, subject to the following conditions:
//
// The above copyright notice and this permission notice (including the next
// paragraph) shall be included in all copies or substantial portions of the
// Software.
//
// THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
// IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS
// FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS
// OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
// WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF
// OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
//
```

Used in chunks 2, 21, 28, 40, 66.

NVIDIA's files follow it with their copyright line:

2 ⟨*NVIDIA license header* 2⟩ ≡

⟨*MIT permission notice* 1⟩

```
// Copyright (c) 2008–2025 NVIDIA Corporation. All rights reserved.
```

Used in chunks 3, 13, 19, 20, 32, 39, 51, 57, 58, 61, 63, 65, 67, 68, 69, 70, 72, 74, 75, 77, 79, 80, 81, 82, 83.

Chapter 2

The Application and Its Scenes

A SwiftUI application is a value of a type that conforms to `App` and describes its *scenes*: windows, and on visionOS immersive spaces. This chapter presents the application type, the model it owns, and two small companions.

2.1 The application

`ViewerApp` imports CloudXRKit unconditionally and Foveated Streaming only when the `FOVEATED_STREAMING` condition is set. That condition, defined in `Config.xcconfig` (Section 5.4), is on for device builds and off for the simulator, which has no Foveated Streaming framework. The three strings name the scenes so that code elsewhere can open and dismiss them.

3 ⟨*CloudXRViewer/Common/ViewerApp.swift* 3⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import CloudXRKit
import SwiftUI
import simd

#if os(visionOS)
// For measuring render camera positions.
import CompositorServices
#endif
#if FOVEATED_STREAMING && os(visionOS)
import FoveatedStreaming
#endif

let launchTitle = "launch"
let contentTitle = "content"
let immersiveTitle = "immersive"
```

Continued in chunks 4, 5, 6, 7, 8, 9, 10, 11, 12. Root chunk, tangled to `CloudXRViewer/Common/ViewerApp.swift`.

The application owns the `AppModel` as state, so the model lives as long as the application does. On visionOS it also keeps the actions that open and dismiss immersive spaces and a user setting that decides whether the launch window stays visible while streaming.

4 ⟨*CloudXRViewer/Common/ViewerApp.swift* 3⟩ +≡

```
@main
struct ViewerApp: App {
    @UIApplicationDelegateAdaptor(AppDelegate.self) var appDelegate
    @Environment(\.scenePhase) var scenePhase
    @Environment(\.openWindow) private var openWindow
    @Environment(\.dismissWindow) private var dismissWindow

    @State private var appModel = AppModel()
    @State var showDisconnectionErrorAlert = false
```

```
#if os(visionOS)
    @Environment(\.dismissImmersiveSpace) private var dismissImmersiveSpace
    @Environment(\.openImmersiveSpace) private var openImmersiveSpace
    @AppStorage("persistLaunchWindow") static var persistLaunchWindow = true
#endif


    // Window sizes
    static let launchSize = CGSize(width: 680, height: 900)
```

Continued in chunks 5, 6, 7, 8, 9, 10, 11, 12.

`CloudXRKit.registerSystems()` registers CloudXRKit's RealityKit systems, which draw the classic stream. Foveated Streaming needs no such registration: the system draws its stream.

5 ⟨*CloudXRViewer/Common/ViewerApp.swift* 3⟩ +≡

```
    init() {
        CloudXRKit.registerSystems()

#if os(visionOS)
        // For tests.
        TestHelper.appModel = appModel


        appDelegate.session = appModel.session
#endif
    }
```

Continued in chunks 6, 7, 8, 9, 10, 11, 12.

2.1.1 Scenes on visionOS

The launch window shows the configuration form. Two observers matter here. The pause sheet modifier, compiled only with Foveated Streaming, lets the system show its own sheet when a foveated session is paused (Section 2.1.2). The `onChange` observer forwards every connection change to the window manager (Chapter 6). It observes `connectionState`, the pair of backend and status, and ignores a change whose backend differs from the previous one: switching the backend picker changes `appModel.status` too, from the status of one session to that of the other, and without the guard a switch back to a backend whose last session had failed would replay its disconnection as if it had just happened.

6 ⟨*CloudXRViewer/Common/ViewerApp.swift* 3⟩ +≡

```
#if os(visionOS)
    var body: some Scene {
        WindowGroup(id: launchTitle) {
            LaunchView(application: $appModel.application)
                .environment(appModel)
                .frame(
                    // Fixed-size window
                    minWidth: Self.launchSize.width,
                    maxWidth: Self.launchSize.width,
                    minHeight: Self.launchSize.height,
                    maxHeight: Self.launchSize.height
                )
                .alert(isPresented: $showDisconnectionErrorAlert) {
                    makeDisconnectionAlert()
                }
#if FOVEATED_STREAMING
                .foveatedStreamingPauseSheet(session: foveatedPauseSheetSession)
#endif
                .onAppear {
                    appModel.windowStateManager.configure(
```

```
                    appModel: appModel,
                    openImmersiveSpace: openImmersiveSpace,
                    dismissImmersiveSpace: dismissImmersiveSpace,
                    openWindow: openWindow,
                    dismissWindow: dismissWindow
                )
                appModel.windowStateManager.windowOnAppear(windowId: launchTitle)
            }
            .onDisappear {
                appModel.windowStateManager.windowOnDisappear(windowId: launchTitle)
            }
            .onChange(of: appModel.connectionState) { oldState, newState in
                guard oldState.backend == newState.backend else { return }
                Task {
                    await appModel.windowStateManager.onConnectionStateChanged(oldState:
↪ oldState.status, newState: newState.status)
                }
            }
    }
    .defaultSize(Self.launchSize)
    .windowResizability(.contentSize)
    .defaultWindowPlacement { content, context in
        if let contentWindow = context.windows.first(
            where: { $0.id == contentTitle }) {
            WindowPlacement(.leading(contentWindow))
        } else {
            WindowPlacement()
        }
    }
```

Continued in chunks 7, 8, 9, 10, 11, 12.

The content window, shown while streaming, holds the HUD, the configuration and the server actions. It observes the connection the same way.

7 ⟨*CloudXRViewer/Common/ViewerApp.swift* 3⟩ +≡

```
#if os(visionOS)
        WindowGroup(id: contentTitle) {
            MainContentView(application: $appModel.application)
                .environment(appModel)
                .frame(
                    // Fixed-size window
                    minWidth: Self.launchSize.width,
                    maxWidth: Self.launchSize.width,
                    minHeight: Self.launchSize.height,
                    maxHeight: Self.launchSize.height
                )
                .onAppear {
                    appModel.windowStateManager.windowOnAppear(windowId: contentTitle)
                }
                .onDisappear {
                    appModel.windowStateManager.windowOnDisappear(windowId: contentTitle)
                }
                .onChange(of: appModel.connectionState) { oldState, newState in
                    guard oldState.backend == newState.backend else { return }
                    Task {
                        await appModel.windowStateManager.onConnectionStateChanged(oldState:
↪ oldState.status, newState: newState.status)
                    }
                }
        }
        .defaultSize(Self.launchSize)
        .windowResizability(.contentSize)
        .defaultWindowPlacement { content, context in
            if let launchWindow = context.windows.first(
```

```
                where: { $0.id == launchTitle }) {
                WindowPlacement(.trailing(launchWindow), width: 1024, height: 600)
            } else {
                WindowPlacement()
            }
        }
#endif  // os(visionOS)
```

Continued in chunks 8, 9, 10, 11, 12.

2.1.2 Two immersive spaces

The classic immersive space hosts `GenericImmersiveView`, which attaches the CloudXRKit session to a RealityKit entity. When the active session disconnects, the application brings back the launch window if it was hidden, dismisses the classic immersive space and, if the disconnection was an error, shows an alert. For the foveated backend the dismissal is left to the session.

8 ⟨*CloudXRViewer/Common/ViewerApp.swift* 3⟩ +≡

```
        ImmersiveSpace(id: immersiveTitle) {

            // Generic viewer.
            if !appModel.application.isConfigurator {
                GenericImmersiveView()
                    .onDisappear {
                        appModel.windowStateManager.immersiveSpaceOnDisappear()
                    }
                    .onAppear {
                        appModel.windowStateManager.immersiveSpaceOnAppear()
                    }
                    .onTapGesture(count: 3) {
                        appModel.windowStateManager.toggleWindow()
                    }.onChange(of: scenePhase) {
                        if scenePhase == .background { // Handle headset doffing during session
                            appModel.windowStateManager.dismissImmersiveSpaceIfOpen()
                        }
                    }
            }
        }
        .environment(appModel)
        .onChange(of: appModel.connectionState) { oldState, newState in
            guard oldState.backend == newState.backend else { return }
            if oldState.status.isDisconnected {
                return
            }
            if newState.status.isDisconnected {
                Task {
                    if !Self.persistLaunchWindow {
                        openWindow(id: launchTitle)
                    }
                    if appModel.backend == .classic {
                        await dismissImmersiveSpace()
                    }
                    alertAndRetryIfError()
                }
            }
        }
```

Continued in chunks 9, 10, 11, 12.

The second immersive space belongs to Foveated Streaming. The initializer `ImmersiveSpace(foveatedStreaming:` makes the system draw the stream in it; the view passed as content, `FoveatedImmersiveView`, only adds overlay content. The space has no identifier: the session opens it with `openImmersiveSpace(foveatedStreaming:)`, which it receives through `immersivePresentationBehaviors`

when connecting (Section 5.3). NVIDIA's guide for the framework uses the progressive immersion style, and so does this scene. Because the scene is built from `appModel.foveatedSession`, that session must exist for the whole life of the application, which is why the model creates it eagerly.

9 ⟨*CloudXRViewer/Common/ViewerApp.swift* 3⟩ +≡

```
#if FOVEATED_STREAMING
        // Foveated streaming. The scene renders the stream itself; the session opens and dismisses
        // it through `immersivePresentationBehaviors`, set in SessionConfigView before connecting.
        ImmersiveSpace(foveatedStreaming: appModel.foveatedSession) {
            FoveatedImmersiveView()
        }
        .environment(appModel)
        .immersionStyle(selection: .constant(.progressive), in: .progressive)
#endif
    }
```

Continued in chunks 10, 11, 12.

The pause sheet modifier takes a binding to an optional session. The binding yields the foveated session only while that backend is selected, so a paused classic session never brings up the system's foveated pause sheet.

10 ⟨*CloudXRViewer/Common/ViewerApp.swift* 3⟩ +≡

```
#if FOVEATED_STREAMING
    /// The session whose system pause sheet is shown; nil while the classic backend is selected.
    var foveatedPauseSheetSession: Binding<FoveatedStreamingSession?> {
        Binding(
            get: { appModel.backend == .foveated ? appModel.foveatedSession : nil },
            set: { _ in }
        )
    }
#endif
```

Continued in chunks 11, 12.

2.1.3 The scene on iOS

On iOS there is a single window. Its root view switches between the configuration form and the streaming view. The window pauses the session when the application goes to the background and resumes it when it becomes active again, through the backend-neutral session.

11 ⟨*CloudXRViewer/Common/ViewerApp.swift* 3⟩ +≡

```
#elseif os(iOS)
    var body: some Scene {
        WindowGroup {
            LaunchView(application: $appModel.application)
                .onAppear {
                    appDelegate.session = appModel.session
                }
                .alert(isPresented: $showDisconnectionErrorAlert) {
                    makeDisconnectionAlert()
                }
                .environment(appModel)
                .onChange(of: appModel.connectionState) { oldState, newState in
                    guard oldState.backend == newState.backend else { return }
                    if oldState.status.isDisconnected {
                        return
                    }
                    if newState.status.isDisconnected {
                        Task {
                            alertAndRetryIfError()
                        }
                    }
```

```
                }
                .onChange(of: scenePhase) {
                    if scenePhase == .background {
                        switch appModel.status {
                        case .connecting, .connected:
                            Task {
                                try? await appModel.activeSession?.pause()
                            }
                        default:
                            return
                        }
                    } else if scenePhase == .active {
                        if appModel.status == .paused {
                            Task {
                                try await appModel.activeSession?.resume()
                            }
                        }
                    }
                }
        }
    }
#endif
```

Continued in chunk 12.

2.1.4 Disconnection alerts

An alert is shown only when the status is a disconnection carrying an error. The message is built from CloudXRKit's detailed error kinds for the classic backend; Foveated Streaming reports a single `DisconnectReason`, whose description becomes the message. The `switch` looks at the CloudXRKit state only while the classic backend is selected, so a stale classic error cannot replace the foveated message.

12 ⟨*CloudXRViewer/Common/ViewerApp.swift* 3⟩ +≡

```
    func alertAndRetryIfError() {
        if case .disconnected(error: .some) = appModel.status {
            showDisconnectionErrorAlert = true
        }

    }

    func makeDisconnectionAlert() -> Alert {
        var errorMessage = "Connection failed without returning an error"
        let debugTips = "Please validate the configuration and, if using a local server instead of GDN,
     ↪ verify that Omniverse is ready."

        // The foveated session reports a single reason.
        if appModel.backend == .foveated, case .disconnected(error: let reason?) = appModel.status {
            errorMessage = "\(reason)\n\nPlease validate the configuration and verify that the streaming
     ↪ server is running and advertising itself."
        }

        // First check appModel session state
        switch appModel.backend == .classic ? appModel.session?.state : nil {
        case .disconnected(let result):
            switch(result) {
            case .failure(let error):
                let description = "Error description: \(error.localizedDescription)"
                switch error {
                case .pauseTimeout:
                    errorMessage = "The session was paused too long and timed out on the server. Please
     ↪ start a new session.\n\n\(description)"
                case .dns:
```

```
                errorMessage = "Error resolving server name. Please verify device's network
    ↪ connection.\n\n\(description)"
            case .failedConnectionAttempt:
                errorMessage = "Connection attempt unsuccessful.\n\n\(debugTips)\n\n\(description)"
            case .sessionTerminatedUnknownReason:
                errorMessage = "Session terminated for an unspecified
    ↪ reason.\n\n\(debugTips)\n\n\(description)"
            case .invalidServerURL:
                errorMessage = "The server URL is invalid, please validate it and try
    ↪ again.\n\n\(description)"
            default:
                errorMessage = "Error type: \(error.kind)\n\n\(description)"
            }
        default:
            errorMessage = "Unknown disconnection reason"
        }
    default:
        break
    }

    return Alert(
        title: Text("Error"),
        message: Text(errorMessage),
        dismissButton: Alert.Button.default(
            Text("OK")
        )
    )
}

}
```

The file ends without a newline.

2.2 The model

`AppModel` holds the state the views share. It imports the foveated library for the platform: the system framework on visionOS, Apple's package on iOS. Both define a type named `FoveatedStreamingSession` with the same interface, which is what lets most of the code ignore the platform. `SavedSettings` keeps the selected application and the selected backend in user defaults.

13 ⟨*CloudXRViewer/Common/AppModel.swift* 13⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import RealityKit
import CloudXRKit
#if FOVEATED_STREAMING
#if os(visionOS)
import FoveatedStreaming
#else
import StreamingSession
#endif
#endif

struct SavedSettings {
    @AppStorage("application") static var application: Application = .generic_viewer
    @AppStorage("streamingBackend") static var backend: StreamingBackend = .classic
}
```

Continued in chunks 14, 15, 16, 17, 18. Root chunk, tangled to `CloudXRViewer/Common/AppModel.swift`.

The model is `@Observable`, so views that read its properties are refreshed when they change, and `@MainActor`, because `FoveatedStreamingSession` is main-actor isolated and the model creates one

as a stored property. The classic `session` is created on the first classic connection, as in NVIDIA's code.

14 ⟨*CloudXRViewer/Common/AppModel.swift* 13⟩ +≡

```
/// The data that the app uses to configure its views.
@MainActor
@Observable
public class AppModel {
    /// The CloudXRKit session, created on the first classic connection.
    public var session: Session?

#if FOVEATED_STREAMING
    /// The foveated streaming session. It lives as long as the app because the visionOS
    /// `ImmersiveSpace(foveatedStreaming:)` scene is built from it.
    let foveatedSession = FoveatedStreamingSession()
#endif
```

Continued in chunks 15, 16, 17, 18.

The saved backend is honored only if this build has it; a simulator build that finds `foveated` in its defaults falls back to `classic`.

15 ⟨*CloudXRViewer/Common/AppModel.swift* 13⟩ +≡

```
    var backend = StreamingBackend.available.contains(SavedSettings.backend) ? SavedSettings.backend :
    ↪ .classic {
        didSet {
            SavedSettings.backend = backend
        }
    }
```

Continued in chunks 16, 17, 18.

`activeSession` is the heart of the backend switch: the session of the selected backend seen through the `ViewerSession` protocol. The classic session is wrapped in a fresh `ClassicSession` on every access; the wrapper holds no state, so this costs nothing and keeps SwiftUI's observation of the underlying CloudXRKit session intact. The foveated session conforms to the protocol directly. `status` and `connectionState` are what most views observe.

16 ⟨*CloudXRViewer/Common/AppModel.swift* 13⟩ +≡

```
    /// The session of the selected backend, if it exists.
    var activeSession: (any ViewerSession)? {
        switch backend {
        case .classic:
            session.map { ClassicSession($0) }
        case .foveated:
#if FOVEATED_STREAMING
            foveatedSession
#else
            nil
#endif
        }
    }

    var status: ViewerStatus {
        activeSession?.viewerStatus ?? .initialized
    }

    var connectionState: ConnectionState {
        ConnectionState(backend: backend, status: status)
    }
```

Continued in chunks 17, 18.

A failed foveated connection may throw before the session reports any status; the error is kept here so the form can show it. `disconnect()` disconnects whichever session is active.

17 ⟨*CloudXRViewer/Common/AppModel.swift* 13⟩ +≡

```
/// Why the last foveated connection attempt failed before the session reported a status.
var foveatedConnectError: String?

func disconnect() async {
    await activeSession?.disconnect()
}
```

Continued in chunk 18.

The rest of the model is NVIDIA's: feedback state, the streaming view flag (now derived from the neutral status), the selected application and, on visionOS, the window manager.

18 ⟨*CloudXRViewer/Common/AppModel.swift* 13⟩ +≡

```
#if DEBUG
    var disableFeedback = true
#else
    var disableFeedback = false
#endif

    var isRatingViewPresented = false
    var ratingText = "Feedback"
    var showDisconnectionAlert = false

    var showStreamingAppView: Bool {
        status == .connected
    }

    var application = SavedSettings.application {
        didSet {
            SavedSettings.application = application
        }
    }

    #if os(visionOS)
    let windowStateManager = WindowStateManager()
    #endif
}
```

The file ends without a newline.

2.3 The application delegate

The delegate exists to disconnect a classic session when the application terminates. It keeps the CloudXRKit session only; a foveated session belongs to the system and ends with the process.

19 ⟨*CloudXRViewer/Common/AppDelegate.swift* 19⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import Foundation
import SwiftUI
import os.log
import CloudXRKit

class AppDelegate: NSObject, UIApplicationDelegate {
    var app: UIApplication?
    var session: Session?

    private static let logger = Logger(
        subsystem: Bundle(for: AppDelegate.self).bundleIdentifier!,
        category: String(describing: AppDelegate.self)
    )
```

```
    func application(
        _ application: UIApplication,
        didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey : Any]? = nil
    ) -> Bool {
        app = application
        return true
    }

    // Currently never called for visionOS
    func applicationWillTerminate(_ application: UIApplication) {
        AppDelegate.logger.info("Disconnecting before app terminates")
        session?.disconnect()
    }
}
```

The file ends without a newline. Root chunk, tangled to `CloudXRViewer/Common/AppDelegate.swift`.

2.4 Test hook

`TestHelper` exposes the model to UI tests.

20 ⟨*CloudXRViewer/Common/TestHelper.swift* 20⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import Foundation

class TestHelper {

    static var appModel: AppModel?
}
```

The file ends without a newline. Root chunk, tangled to `CloudXRViewer/Common/TestHelper.swift`.

Chapter 3

A Vocabulary Shared by Both Backends

This chapter is the single file `ViewerSession.swift`. It names the backends and defines the types that let the user interface work with either library: a status, a channel identifier, a channel and a session protocol. None of them imports CloudXRKit or Foveated Streaming.

3.1 Backends

The enumeration names the backends and gives each a label for the picker. On iOS the foveated case is called “Streaming Session” because an iPhone has no eye tracking and the stream is not foveated there.

21 ⟨*CloudXRViewer/Common/Streaming/ViewerSession.swift* 21⟩ ≡
⟨*MIT permission notice* 1⟩

```
// Copyright (c) 2026 Pablo Figueroa and Claude.

import Foundation

/// The streaming stack used to reach the server.
///
/// `classic` is the CloudXRKit Swift package this viewer was built on. `foveated` is Apple's
/// Foveated Streaming system framework on visionOS (the headset shares the approximate region the
/// user looks at so the server spends its bits there) and Apple's StreamingSession package on iOS,
/// which exposes the same API without eye tracking.
enum StreamingBackend: String, CaseIterable {
    case classic
    case foveated

    var label: String {
        switch self {
        case .classic:
            "CloudXR"
#if os(visionOS)
        case .foveated:
            "Foveated Streaming"
#else
        case .foveated:
            "Streaming Session"
#endif
        }
    }
```

Continued in chunks 22, 23, 24, 25, 26, 27. Root chunk, tangled to `CloudXRViewer/Common/Streaming/ViewerSession.swift`.

`available` is the list the configuration form offers. Without `FOVEATED_STREAMING`, that is in a simulator build, only the classic backend exists, and the form hides the picker.

22 ⟨*CloudXRViewer/Common/Streaming/ViewerSession.swift* 21⟩ +≡

```
    /// The backends this build can use; simulator builds have no foveated backend.
    static var available: [StreamingBackend] {
#if FOVEATED_STREAMING
        allCases
#else
        [.classic]
#endif
    }
}
```

Continued in chunks 23, 24, 25, 26, 27.

3.2 Status

`ViewerStatus` is the union of the two libraries' state machines, reduced to what the views need. CloudXRKit's `authenticating` and `authenticated` states fold into `connecting`; both libraries' disconnection payloads become an optional error description, `nil` for a clean disconnection. Two helpers answer the questions the views ask most.

23 ⟨*CloudXRViewer/Common/Streaming/ViewerSession.swift* 21⟩ +≡

```
/// Connection status shared by both backends, so the UI does not depend on either SDK's enum.
enum ViewerStatus: Equatable {
    case initialized
    case connecting
    case connected
    case pausing
    case paused
    case resuming
    case disconnecting
    /// `error` is nil when the disconnection was clean.
    case disconnected(error: String?)

    /// The session can start a new connection.
    var isIdle: Bool {
        switch self {
        case .initialized, .disconnected:
            true
        default:
            false
        }
    }

    var isDisconnected: Bool {
        if case .disconnected = self { true } else { false }
    }
}
```

Continued in chunks 24, 25, 26, 27.

`ConnectionState` pairs the status with its backend. Observers compare consecutive values and ignore changes caused by switching backends (Section 2).

24 ⟨*CloudXRViewer/Common/Streaming/ViewerSession.swift* 21⟩ +≡

```
/// A status together with the backend that reported it, so observers can tell a real status
/// change from one caused by switching backends.
struct ConnectionState: Equatable {
    let backend: StreamingBackend
    let status: ViewerStatus
}
```

Continued in chunks 25, 26, 27.

3.3 Channels

An opaque data channel is a bidirectional pipe for application messages between client and server. CloudXRKit identifies a channel by a `ChannelInfo`, Foveated Streaming by a `MessageChannel.ID`. `ViewerChannelID` carries either one as an `AnyHashable` key, plus a label for the picker; each adapter casts the key back to its own type.

25 ⟨*CloudXRViewer/Common/Streaming/ViewerSession.swift* 21⟩ +≡

```
/// Identifies an opaque data channel independently of the backend's own channel type.
struct ViewerChannelID: Hashable {
    /// Human-readable name for the channel picker.
    let label: String
    /// The backend's own identifier (`ChannelInfo` or `FoveatedStreamingSession.MessageChannel.ID`).
    let key: AnyHashable
}
```

Continued in chunks 26, 27.

A channel delivers messages as an asynchronous stream and sends with a throwing call. CloudXRKit's `MessageChannel` reports a failed send by returning `false`; its adapter turns that into `ChannelSendError`.

26 ⟨*CloudXRViewer/Common/Streaming/ViewerSession.swift* 21⟩ +≡

```
/// An open opaque data channel to the server.
protocol ViewerChannel: AnyObject {
    var receivedMessageStream: AsyncStream<Data> { get }
    var statusDescription: String { get }
    func send(_ data: Data) throws
}

struct ChannelSendError: LocalizedError {
    var errorDescription: String? { "The message could not be sent on this channel" }
}
```

Continued in chunk 27.

3.4 The session protocol

`ViewerSession` is what the views use: status, a description to show, the channels, and the lifecycle calls. The lifecycle calls are `async` because Foveated Streaming's are; CloudXRKit's synchronous calls satisfy them trivially. The protocol is main-actor isolated, like `FoveatedStreamingSession`, so the foveated session can conform without any wrapper.

27 ⟨*CloudXRViewer/Common/Streaming/ViewerSession.swift* 21⟩ +≡

```
/// What the views need from a streaming session, whichever backend provides it.
@MainActor
protocol ViewerSession: AnyObject {
    var viewerStatus: ViewerStatus { get }
    /// The backend's own status text, shown under the configuration form.
    var statusDescription: String { get }
    var channelIDs: [ViewerChannelID] { get }
    func channel(for id: ViewerChannelID) -> (any ViewerChannel)?
    func disconnect() async
    func pause() async throws
    func resume() async throws
}
```

Chapter 4

The CloudXRKit Backend

The classic backend is NVIDIA's original code path. This chapter adapts its session to `ViewerSession` and then presents the configuration form's logic, which connects either backend.

4.1 The adapter

`ClassicSession` wraps a CloudXRKit `Session`. It stores nothing but the session, so it can be created on demand (Section 2.2). The status mapping is a direct translation; the `@unknown default` is required because CloudXRKit is built with library evolution and may add states in a later version.

28 ⟨*CloudXRViewer/Common/Streaming/ClassicSession.swift* 28⟩ ≡
⟨*MIT permission notice* 1⟩

```
// Copyright (c) 2026 Pablo Figueroa and Claude.

import CloudXRKit
import Foundation

/// Adapts a CloudXRKit `Session` to `ViewerSession`.
///
/// The adapter keeps no state of its own: every property reads through to the wrapped session, so
/// SwiftUI observation of `Session.state` keeps working through it.
@MainActor
final class ClassicSession: ViewerSession {
    let session: Session

    init(_ session: Session) {
        self.session = session
    }

    var viewerStatus: ViewerStatus {
        switch session.state {
        case .initialized:
            .initialized
        case .authenticating, .authenticated, .connecting:
            .connecting
        case .connected:
            .connected
        case .pausing:
            .pausing
        case .paused:
            .paused
        case .resuming:
            .resuming
        case .disconnecting:
            .disconnecting
        case .disconnected(let result):
```

```
            switch result {
            case .success:
                .disconnected(error: nil)
            case .failure(let error):
                .disconnected(error: error.localizedDescription)
            }
        @unknown default:
            .initialized
        }
    }
```

Continued in chunks 29, 30, 31. Root chunk, tangled to `CloudXRViewer/Common/Streaming/ClassicSession.swift`.

CloudXRKit names a channel by a `ChannelInfo` whose `uuid` is a byte array; the label prints those bytes as NVIDIA's picker did.

29 ⟨*CloudXRViewer/Common/Streaming/ClassicSession.swift* 28⟩ +≡

```
    var statusDescription: String {
        session.state.description
    }

    var channelIDs: [ViewerChannelID] {
        session.availableMessageChannels.map { channelInfo in
            ViewerChannelID(
                label: "Channel [\(channelInfo.uuid.map { String($0) }.joined(separator: ","))]",
                key: channelInfo
            )
        }
    }

    func channel(for id: ViewerChannelID) -> (any ViewerChannel)? {
        guard let channelInfo = id.key.base as? ChannelInfo else {
            return nil
        }
        return session.getMessageChannel(channelInfo)
    }
```

Continued in chunks 30, 31.

The lifecycle calls forward to CloudXRKit's synchronous versions.

30 ⟨*CloudXRViewer/Common/Streaming/ClassicSession.swift* 28⟩ +≡

```
    func disconnect() async {
        session.disconnect()
    }

    func pause() async throws {
        session.pause()
    }

    func resume() async throws {
        try session.resume()
    }
}
```

Continued in chunk 31.

CloudXRKit's channel class conforms to `ViewerChannel` through an extension, so no wrapper object is needed.

31 ⟨*CloudXRViewer/Common/Streaming/ClassicSession.swift* 28⟩ +≡

```
extension CloudXRKit.MessageChannel: ViewerChannel {
    var statusDescription: String {
        status.rawValue
    }

    func send(_ data: Data) throws {
        guard sendServerMessage(data) else {
            throw ChannelSendError()
        }
    }
}
```

4.2 The configuration form

`SessionConfigView` is the form shown in the launch window. Its layout is platform specific (Chapters 6 and 7); this file holds the state and the connection logic. Most settings are stored in user defaults. The three foveated settings are new; the manual IP address field is shared by both backends.

32 ⟨*CloudXRViewer/Common/SessionConfigView.swift* 32⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import CloudXRKit
import OSLog
#if FOVEATED_STREAMING
#if os(visionOS)
import FoveatedStreaming
#else
import StreamingSession
#endif
#endif

struct SessionConfigView: View {
    @AppStorage("hostAddress") var hostAddress: String = ""
    @AppStorage("zone") private(set) var zone: Zone = .us_west
    @AppStorage("authMethod") private(set) var authMethod: AuthMethod = .starfleet
    @AppStorage("resolutionPreset") var resolutionPreset: ResolutionPreset = .standardPreset

#if os(visionOS)
    @AppStorage("enableHandTracking") var enableHandTracking: Bool = false
#endif

    @AppStorage("genericAppID") var genericAppID: Int = 0

    // Foveated streaming endpoint; the manual IP address reuses `hostAddress`.
    @AppStorage("foveatedEndpointKind") var foveatedEndpointKind: FoveatedEndpointKind = .discover
    @AppStorage("foveatedPort") var foveatedPort: Int = FoveatedEndpointKind.defaultPort
    @AppStorage("remoteServerName") var remoteServerName: String = ""
```

Continued in chunks 33, 34, 35, 36, 37, 38. Root chunk, tangled to `CloudXRViewer/Common/SessionConfigView.swift`.

On visionOS the form needs both immersive space actions: the foveated session receives them to present and dismiss its space.

33 ⟨ *CloudXRViewer/Common/SessionConfigView.swift* 32⟩ +≡

```
    @Environment(AppModel.self) var appModel
    @Environment(\.colorScheme) var colorScheme

#if os(visionOS)
    @Environment(\.openImmersiveSpace) var openImmersiveSpace
    @Environment(\.dismissImmersiveSpace) var dismissImmersiveSpace
    // TODO(tifchen): Revisit whether this can be included for iOS.
#endif

    @State var guestModeComponent: GuestModeComponent = GuestModeComponent()

    @State var awsalb = ""
    @State var sessionId = ""

    @State var useSimpleConfigView: Bool = {
        false
    }()

    @Binding var application: Application
    @State var isGuestModePopoverPresented: Bool = false
```

Continued in chunks 34, 35, 36, 37, 38.

`sessionConnected` disables settings that cannot change during a session; it reads the neutral status, so it works for both backends.

34 ⟨ *CloudXRViewer/Common/SessionConfigView.swift* 32⟩ +≡

```
    var sessionConnected: Bool {
        switch appModel.status {
        case .initialized, .disconnected, .paused:
            false
        default:
            true
        }
    }

    private static let logger = Logger(
        subsystem: Bundle.main.bundleIdentifier!,
        category: String(describing: SessionConfigView.self)
    )

    // The order of these vars is important - the completion handler should be at the end
    var completionHandler: () -> Void
```

Continued in chunks 35, 36, 37, 38.

The main view stacks the form, the status line and the connect button. The guest-mode popover is part of NVIDIA's GDN sign-in flow.

35 ⟨ *CloudXRViewer/Common/SessionConfigView.swift* 32⟩ +≡

```
    var mainView: some View {
        VStack(alignment: .leading, spacing: 0) {
            ScrollView {
                VStack(spacing: 16) {
                    if useSimpleConfigView {
                        simpleConfigView
                    } else {
                        sessionConfigForm
                    }
                }
                .padding(.top, 8)
            }

            VStack(spacing: 16) {
```

```
                Text(stateDescription)
                    .frame(maxWidth: .infinity, alignment: .center)

                connectButton
            }
            .padding(.vertical, 128)
            .background(
                Color(.systemBackground)
                    .opacity(0.95)
                    .ignoresSafeArea(edges: .bottom)
            )
        }
        .popover(isPresented: $isGuestModePopoverPresented,
                 attachmentAnchor: .point(.center),
                 arrowEdge: .top) {
            VStack {
                guestModeComponent.makeView()
            }
            .onAppear {
                guestModeComponent.configure {
                    isGuestModePopoverPresented = false
                    doConnect()
                }
            }
        }
        .presentationCompactAdaptation(.popover)
    }

    @ViewBuilder
    private var simpleConfigView: some View {
        VStack(alignment: .center) {
            Text("NVIDIA Omniverse Configurator")
                .font(.title)
                .padding(.top, 240)
        }
        .onAppear {
            zone = .us_west
            authMethod = .guest
            resolutionPreset = .standardPreset
            application = .generic_viewer
        }
        .frame(maxWidth: .infinity, minHeight: 360)
    }
```

Continued in chunks 36, 37, 38.

The connect button doubles as resume and disconnect button. Disconnecting goes through `appModel.disconnect()`, so the same dialog serves both backends.

36 ⟨ *CloudXRViewer/Common/SessionConfigView.swift* 32⟩ +≡

```
    private var connectButton: some View {
        Button(buttonLabel) {
            handleConnectButtonTap()
        }
        .disabled(connectButtonDisabled)
        .sheet(isPresented: Binding(get: { appModel.isRatingViewPresented }, set: { _ in })) {
            StarRatingView()
        }
        .confirmationDialog(
            "Do you really want to disconnect?",
            isPresented: Binding(
                get: { appModel.showDisconnectionAlert },
                set: { appModel.showDisconnectionAlert = $0 }
            ),
            titleVisibility: .visible
```

```
        ) {
            if !appModel.disableFeedback {
                Button("Disconnect with feedback") {
                    Task { await appModel.disconnect() }
                    appModel.isRatingViewPresented = true
                }
                Button("Disconnect without feedback", role: .destructive) {
                    Task { await appModel.disconnect() }
                }
            } else {
                Button("Disconnect", role: .destructive) {
                    Task { await appModel.disconnect() }
                }
            }
            Button("Cancel", role: .cancel) {}
        }
    }
```

Continued in chunks 37, 38.

4.3 Connecting with CloudXRKit

A tap is routed to the foveated flow when that backend is selected. Otherwise NVIDIA's logic runs: resume a paused session, ask before disconnecting a connected one, and otherwise start a connection, through the guest sign-in popover when needed.

37 ⟨*CloudXRViewer/Common/SessionConfigView.swift* 32⟩ +≡

```
    private func handleConnectButtonTap() {
#if FOVEATED_STREAMING
        if appModel.backend == .foveated {
            handleFoveatedConnectButtonTap()
            return
        }
#endif

        if appModel.session?.state == .paused {
            try! appModel.session?.resume()
            self.completionHandler()
            return
        }

        if appModel.session?.state == .connected {
            appModel.showDisconnectionAlert = true
            return
        }

        // To be safe, disconnect from previous session
        switch appModel.session?.state {
        case .disconnected, .disconnecting:
            break
        default:
            appModel.session?.disconnect()
        }

        if usingGuestMode {
            isGuestModePopoverPresented = true
        }
        else {
            doConnect()
        }
    }
```

Continued in chunk 38.

`doConnect()` builds a `CloudXRKit.Config`: resolution preset, hand-tracking mode (simulated in the simulator), and the connection type, either NVIDIA's Graphics Delivery Network with an application identifier or a local server by IP address. The session is created on first use and reconfigured on later connections. The foveated counterpart, the `foveated connect flow` chunk referenced at the end of the struct, is presented in Section 5.3.

38 ⟨*CloudXRViewer/Common/SessionConfigView.swift* 32⟩ +≡

```
    private func doConnect() {
        let preset = resolutionPreset
        var cxrConfig = CloudXRKit.Config()
        cxrConfig.resolutionPreset = preset

#if os(visionOS)
    #if targetEnvironment(simulator)
        cxrConfig.handTrackingMode = enableHandTracking ? .simulated : .disabled
    #else
        cxrConfig.handTrackingMode = enableHandTracking ? .prediction : .disabled
    #endif
#endif

        var appID: UInt = 0

        if zone != .ipAddress {
            if application.appID != .unknown {
                appID = application.appID.rawValue
            } else if genericAppID > 0 {
                appID = UInt(genericAppID)
            } else {
                Self.logger.error("No valid appID configured for this application.")
                return
            }

            if !usingGuestMode {
                cxrConfig.connectionType = .nvGraphicsDeliveryNetwork(
                    appId: UInt(appID),
                    authenticationType: .starfleet(),
                    zone: zone.id
                )
            }
        } else {
            cxrConfig.connectionType = .local(ip: hostAddress)
        }

        if appModel.session == nil {
            appModel.session = CloudXRSession(config: cxrConfig)
        }

        Task { @MainActor in
            if usingGuestMode {
                let guestAuth = try await guestModeComponent.getGuestAuth(appID: appID)

                cxrConfig.connectionType = .nvGraphicsDeliveryNetwork(
                    appId: UInt(appID),
                    authenticationType: guestAuth,
                    zone: zone.id)
            }

            appModel.session?.configure(config: cxrConfig)
            try await appModel.session?.connect()
            completionHandler()
        }
    }
```

```
⟨foveated connect flow 46⟩
}

#Preview {
    @Previewable @State var appModel = AppModel()
    return SessionConfigView(application: $appModel.application) { () -> Void in }
        .environment(appModel)
}
```

4.4 Guest mode

The guest sign-in component is a stub in the public version of NVIDIA's viewer: it hosts a `UIView` for an authentication page but does not implement the flow.

39 ⟨*CloudXRViewer/Common/GuestMode.swift* 39⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import CloudXRKit
import OSLog

// MARK: - Combined Guest Mode Component
@Observable class GuestModeComponent {
    // MARK: - UIViewRepresentable for the combined component
    struct AuthRepresentableView: UIViewRepresentable {
        let uiView: UIView

        func makeUIView(context: Context) -> UIView {
            return uiView
        }

        func updateUIView(_ view: UIView, context: Context) {
            // nothing to update
        }
    }

    // MARK: - Observable Properties
    var token: String?

    // MARK: - Private Properties
    let width: CGFloat = 640
    let height: CGFloat = 480
    let uiView = UIView()

    private static let logger = Logger(
        subsystem: Bundle.main.bundleIdentifier!,
        category: String(describing: GuestModeComponent.self)
    )

    init() {
        self.uiView.backgroundColor = .clear
    }

    func makeView() -> some View {
        AuthRepresentableView(uiView: uiView)
            .frame(width: width, height: height, alignment: .center)
    }

    func configure(onSuccess: @escaping () -> Void) {
        Self.logger.error("Guest mode not implemented")
    }
```

```
    func getGuestAuth(appID: UInt) async throws -> AuthenticationType {
        fatalError("Guest mode not implemented")
    }
}
```

Root chunk, tangled to `CloudXRViewer/Common/GuestMode.swift`.

Chapter 5

The Foveated Streaming Backend

On visionOS 26.4 and later, Foveated Streaming is a system framework: there is no package to add, only `import FoveatedStreaming`, an entitlement, and a server that speaks Apple's session protocol. On iOS, Apple's StreamingSession package offers the same types without the eye tracking. This chapter adapts the session, presents the connection flow, and then the build configuration.

5.1 Endpoints

A foveated session connects to an *endpoint*. `FoveatedEndpointKind` lists the three kinds the form offers:

System discovery
: The system browses the local network for servers advertising `_apple-foveated-streaming._tcp`, lets the user pick one, and pairs with it by scanning a QR code the server displays; the code carries a client token and the server's certificate fingerprint.

Local address
: A server at a known IP address and port; the session-management protocol listens on port 55000 by default.

Remote server
: A cloud deployment reached by name through a signaling service.

The enumeration and the default port are compiled in every build because the form's stored settings use them; everything that touches the framework is inside `FOVEATED_STREAMING`.

40 ⟨*CloudXRViewer/Common/Streaming/FoveatedSession.swift* 40⟩ ≡
⟨*MIT permission notice* 1⟩

```
// Copyright (c) 2026 Pablo Figueroa and Claude.

import Foundation
import Network
#if FOVEATED_STREAMING
#if os(visionOS)
import FoveatedStreaming
#else
import StreamingSession
#endif
#endif

/// How the foveated session finds its server.
enum FoveatedEndpointKind: String, CaseIterable {
    /// The system browses for `_apple-foveated-streaming._tcp` servers and pairs with a QR code.
    case discover = "Discover & pair (QR code)"
    /// A server on the local network at a known address.
```

```
    case local = "Manual IP address"
    /// A cloud server reached through a signaling service.
    case remote = "Remote server"
}

extension FoveatedEndpointKind {
    /// The default port of the foveated streaming session-management protocol.
    static let defaultPort = 55000
}
```

Continued in chunks 41, 42, 43, 44, 45. Root chunk, tangled to `CloudXRViewer/Common/Streaming/FoveatedSession.swift`.

`endpoint(hostAddress:port:serverName:)` turns the form's fields into the framework's `Endpoint`, validating the address (IPv4 or IPv6), the port and the server name. Errors carry a message the form can show.

41 ⟨*CloudXRViewer/Common/Streaming/FoveatedSession.swift* 40⟩ +≡

```
#if FOVEATED_STREAMING
struct FoveatedEndpointError: LocalizedError {
    let errorDescription: String?
}

extension FoveatedEndpointKind {

    func endpoint(hostAddress: String, port: Int, serverName: String) throws ->
    ↪ FoveatedStreamingSession.Endpoint {
        switch self {
        case .discover:
            return .systemDiscovered
        case .local:
            let host = hostAddress.trimmingCharacters(in: .whitespacesAndNewlines)
            let ipAddress: any IPAddress
            if let ipv4 = IPv4Address(host) {
                ipAddress = ipv4
            } else if let ipv6 = IPv6Address(host) {
                ipAddress = ipv6
            } else {
                throw FoveatedEndpointError(errorDescription: "\"\(host)\" is not a valid IP address")
            }
            guard (1...Int(UInt16.max)).contains(port), let nwPort = NWEndpoint.Port(rawValue:
    ↪ UInt16(port)) else {
                throw FoveatedEndpointError(errorDescription: "\(port) is not a valid port")
            }
            return .local(ipAddress: ipAddress, port: nwPort)
        case .remote:
            let name = serverName.trimmingCharacters(in: .whitespacesAndNewlines)
            guard !name.isEmpty else {
                throw FoveatedEndpointError(errorDescription: "Enter a server name")
            }
            return .remote(serverName: name, signalingHeaders: [:])
        }
    }
}
```

Continued in chunks 42, 43, 44, 45.

5.2 The session

Unlike CloudXRKit's session, `FoveatedStreamingSession` conforms to `ViewerSession` directly: it is already main-actor isolated and already has `disconnect()`, `pause()` and `resume()` with the protocol's signatures, so the extension only adds the status mapping, the description and the channels. A disconnection the application asked for is not an error; `endpointInitiatedDisconnect`, `unauthorized` and any other reason are.

42 ⟨*CloudXRViewer/Common/Streaming/FoveatedSession.swift* 40⟩ +≡

```
extension FoveatedStreamingSession: ViewerSession {
    var viewerStatus: ViewerStatus {
        switch status {
        case .initialized:
            .initialized
        case .connecting:
            .connecting
        case .connected:
            .connected
        case .pausing:
            .pausing
        case .paused:
            .paused
        case .resuming:
            .resuming
        case .disconnecting:
            .disconnecting
        case .disconnected(let reason):
            .disconnected(error: reason == .appInitiatedDisconnect ? nil : reason.localizedDescription)
        @unknown default:
            .initialized
        }
    }
```

Continued in chunks 43, 44, 45.

The framework returns the channel identifiers as a set on visionOS and as an array on iOS; sorting the mapped identifiers by label gives the picker a stable order on both.

43 ⟨*CloudXRViewer/Common/Streaming/FoveatedSession.swift* 40⟩ +≡

```
    var statusDescription: String {
        status.description
    }

    var channelIDs: [ViewerChannelID] {
        availableMessageChannels
            .map { ViewerChannelID(label: "Channel \($0.label)", key: $0) }
            .sorted { $0.label < $1.label }
    }

    func channel(for id: ViewerChannelID) -> (any ViewerChannel)? {
        guard let channelID = id.key.base as? MessageChannel.ID else {
            return nil
        }
        return messageChannel(for: channelID)
    }
}
```

Continued in chunks 44, 45.

The text form of a channel identifier, `stringRepresentation`, appeared in visionOS 27. The deployment target is visionOS 26.4, the first release with Foveated Streaming, so on older systems the label falls back to the identifier's description. The iOS package has the property without an availability limit.

44 ⟨*CloudXRViewer/Common/Streaming/FoveatedSession.swift* 40⟩ +≡

```
private extension FoveatedStreamingSession.MessageChannel.ID {
    var label: String {
#if os(visionOS)
        // visionOS 26 has no public text form of a channel ID.
        if #available(visionOS 27.0, *) {
            return stringRepresentation
        }
        return String(describing: self)
```

```
#else
        return stringRepresentation
#endif
    }
}
```

Continued in chunk 45.

The framework's channel already has the stream and a throwing send; the extension adds the description.

45 ⟨*CloudXRViewer/Common/Streaming/FoveatedSession.swift* 40⟩ +≡

```
extension FoveatedStreamingSession.MessageChannel: ViewerChannel {
    var statusDescription: String {
        switch channelStatus {
        case .ready:
            "ready"
        case .closed:
            "closed"
        @unknown default:
            "unknown"
        }
    }

    func send(_ data: Data) throws {
        try sendMessage(data)
    }
}
#endif
```

5.3 Connecting

This chunk completes `SessionConfigView` (Chapter 4). A tap on the button resumes a paused session, asks before disconnecting a connected one, and otherwise connects.

Connecting first builds the endpoint. On visionOS it then hands the session the two immersive space actions through `immersivePresentationBehaviors`: with `.automatic`, the session opens its immersive space when streaming starts, dismisses it when the session pauses or disconnects, and presents it again on resume. This is the main behavioral difference from CloudXRKit, where the application opens its immersive space after connecting. Letting the session do it keeps the space in step with the stream, including transitions the application does not initiate, such as a server closing the session; correspondingly, the launch view and the window manager open and dismiss the classic space only (Chapter 6).

Hand tracking is an input capability the session must request. The capability type was added in visionOS 27, so the request is made only there, after asking the user for authorization. Then the session connects; any error, thrown by the endpoint validation or by the framework, is logged and shown under the form.

46 ⟨*foveated connect flow* 46⟩ ≡

```
#if FOVEATED_STREAMING
    private func handleFoveatedConnectButtonTap() {
        let session = appModel.foveatedSession

        switch session.status {
        case .paused:
            Task { @MainActor in
                do {
                    try await session.resume()
                    completionHandler()
```

```
                } catch {
                    Self.logger.error("Foveated streaming resume failed: \(error.localizedDescription)")
                    appModel.foveatedConnectError = error.localizedDescription
                }
            }
        case .connected:
            appModel.showDisconnectionAlert = true
        default:
            doConnectFoveated()
        }
    }

    private func doConnectFoveated() {
        let session = appModel.foveatedSession
        appModel.foveatedConnectError = nil

        Task { @MainActor in
            do {
                let endpoint = try foveatedEndpointKind.endpoint(
                    hostAddress: hostAddress,
                    port: foveatedPort,
                    serverName: remoteServerName
                )

#if os(visionOS)
                // The session opens the immersive space once streaming starts and dismisses it
                // on disconnect, so neither LaunchView nor WindowStateManager opens it.
                session.immersivePresentationBehaviors = .automatic(openImmersiveSpace,
    ↪ dismissImmersiveSpace)

                if #available(visionOS 27.0, *) {
                    let capabilities: Set<FoveatedStreamingSession.InputCapability> = enableHandTracking
    ↪ ? [.handTracking] : []
                    if !capabilities.isEmpty {
                        _ = await session.requestAuthorization(for: capabilities)
                    }
                    session.requestedInputCapabilities = capabilities
                }
#endif

                try await session.connect(endpoint: endpoint)
                completionHandler()
            } catch {
                Self.logger.error("Foveated streaming connection failed: \(error.localizedDescription)")
                appModel.foveatedConnectError = error.localizedDescription
            }
        }
    }
#endif
```

Used in chunk 38.

5.4 Build configuration

Neither library ships for the simulator: the visionOS simulator SDK has no FoveatedStreaming framework, and Apple's xcframework has only a device slice. The fork keeps NVIDIA's ability to run the classic backend in the visionOS simulator by compiling the foveated code only for device SDKs. The condition `FOVEATED_STREAMING` is set with SDK-conditional build settings in `Config.xcconfig`, which all targets use as their base configuration; the `$(inherited)` keeps the `DEBUG` condition the targets set themselves. The iOS target cannot be built for the iOS simulator at all, because the StreamingSession package is linked unconditionally; NVIDIA's iOS client needs

ARKit's spatial tracking, which the simulator lacks anyway.

The file also sets the version, now 6.1.0, and the bundle identifier.

47 ⟨*Config.xcconfig* 47⟩ ≡

```
//
//  Config.xcconfig
//  CloudXR Viewer
//

// Configuration settings file format documentation can be found at:
// https://developer.apple.com/documentation/xcode/adding-a-build-configuration-file-to-your-project

// Build configuration
MARKETING_VERSION = 6.1.0
PRODUCT_BUNDLE_IDENTIFIER = com.nvidia.cloudxr-viewer

// Apple's FoveatedStreaming framework (visionOS) and StreamingSession package (iOS) ship for
// devices only, so the foveated backend is compiled into device builds only.
SWIFT_ACTIVE_COMPILATION_CONDITIONS[sdk=xros*] = $(inherited) FOVEATED_STREAMING
SWIFT_ACTIVE_COMPILATION_CONDITIONS[sdk=iphoneos*] = $(inherited) FOVEATED_STREAMING
```

Root chunk, tangled to `Config.xcconfig`.

5.5 Entitlements and property lists

The visionOS target's entitlements add `com.apple.developer.foveated-streaming-session` to the low-latency streaming entitlement NVIDIA already used. Both are granted to members of the Apple Developer Program.

48 ⟨*CloudXRViewer/visionOS/CloudXRViewer-visionOS.entitlements* 48⟩ ≡

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>com.apple.developer.foveated-streaming-session</key>
    <true/>
    <key>com.apple.developer.low-latency-streaming</key>
    <true/>
</dict>
</plist>
```

Root chunk, tangled to `CloudXRViewer/visionOS/CloudXRViewer-visionOS.entitlements`.

System discovery browses for the Bonjour service type of Apple's session protocol. An application that browses the local network must declare the types it uses in `NSBonjourServices`, so both property lists name `_apple-foveated-streaming._tcp`. The remaining keys are NVIDIA's: usage descriptions for hand tracking and world sensing, file sharing, and two application identifiers filled in from build settings.

The visionOS property list:

49 ⟨*CloudXRViewer/visionOS/CloudXRViewer-visionOS-Info.plist* 49⟩ ≡

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>NSBonjourServices</key>
    <array>
        <string>_apple-foveated-streaming._tcp</string>
    </array>
    <key>NSHandsTrackingUsageDescription</key>
    <string>Enables sending hand positions to a server.</string>
    <key>NSWorldSensingUsageDescription</key>
```

```
    <string>Enables placing objects.</string>
    <key>UIFileSharingEnabled</key>
    <true/>
    <key>NISSAN_CONFIGURATOR_APP_ID</key>
    <string>$(NISSAN_CONFIGURATOR_APP_ID)</string>
    <key>PURSE_CONFIGURATOR_APP_ID</key>
    <string>$(PURSE_CONFIGURATOR_APP_ID)</string>
</dict>
</plist>
```

Root chunk, tangled to `CloudXRViewer/visionOS/CloudXRViewer-visionOS-Info.plist`.

The iOS property list. The camera, local network and microphone usage descriptions are generated from build settings in the project file.

50 ⟨*CloudXRViewer/iOS/CloudXRViewer-iOS-Info.plist* 50⟩ ≡

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>NSBonjourServices</key>
    <array>
        <string>_apple-foveated-streaming._tcp</string>
    </array>
    <key>UIFileSharingEnabled</key>
    <true/>
    <key>NISSAN_CONFIGURATOR_APP_ID</key>
    <string>$(NISSAN_CONFIGURATOR_APP_ID)</string>
    <key>PURSE_CONFIGURATOR_APP_ID</key>
    <string>$(PURSE_CONFIGURATOR_APP_ID)</string>
</dict>
</plist>
```

Root chunk, tangled to `CloudXRViewer/iOS/CloudXRViewer-iOS-Info.plist`.

Chapter 6

The visionOS User Interface

On visionOS the application has two windows, a launch window with the configuration form and a content window shown while streaming, and the two immersive spaces of Chapter 2. A window manager keeps them consistent with the connection.

6.1 The window manager

`WindowStateManager` is NVIDIA's coordinator of windows and immersive spaces. It is now main-actor isolated: it reads the main-actor model, and all its callers are views. That also made the lock around `pauseOnImmersiveSpaceDismissed` unnecessary, so the property is a plain variable.

51 ⟨*CloudXRViewer/Common/WindowStateManager.swift* 51⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import os.log

#if os(visionOS)
// Manages the state and lifecycle of windows and immersive spaces in the application
@MainActor
class WindowStateManager {
    // Logger instance for debugging and error tracking
    private static let logger = Logger()

    // Current state of windows visibility
    private var visibleWindows: Set<String> = []

    // Current state of the immersive space
    private(set) var immersiveSpaceActive = false

    // Indicates if the manager has been configured with necessary actions
    private var configured = false

    // Reference to the appModel (weak to avoid retain cycle)
    private weak var appModel: AppModel!

    // Actions for managing windows and immersive spaces
    private var openImmersiveSpace: OpenImmersiveSpaceAction!
    private var dismissImmersiveSpace: DismissImmersiveSpaceAction!
    private var openWindow: OpenWindowAction!
    private var dismissWindow: DismissWindowAction!

    // When the immersive space is dismissed, we want to disconnect, but in the case of the IPD check,
    ↪ we do not.
    // The manager is main-actor isolated, so this needs no lock.
    var pauseOnImmersiveSpaceDismissed = true
```

Continued in chunks 52, 53, 54, 55, 56. Root chunk, tangled to `CloudXRViewer/Common/WindowStateManager.swift`.

The manager receives the environment's actions once, from the launch window.

52 ⟨*CloudXRViewer/Common/WindowStateManager.swift* 51⟩ +≡

```
// Initializes the manager with necessary actions and session reference.
// Should be called at least once before any other functions.
// - Parameters:
//   - appModel: The app model instance
//   - openImmersiveSpace: Action to open immersive space
//   - dismissImmersiveSpace: Action to dismiss immersive space
//   - openWindow: Action to open window
//   - dismissWindow: Action to dismiss window
func configure(appModel: AppModel,
               openImmersiveSpace: OpenImmersiveSpaceAction,
               dismissImmersiveSpace: DismissImmersiveSpaceAction,
               openWindow: OpenWindowAction,
               dismissWindow: DismissWindowAction) {
    self.appModel = appModel
    self.openImmersiveSpace = openImmersiveSpace
    self.dismissImmersiveSpace = dismissImmersiveSpace
    self.openWindow = openWindow
    self.dismissWindow = dismissWindow

    configured = true
}
```

Continued in chunks 53, 54, 55, 56.

Only one window is visible at a time: showing one dismisses the others.

53 ⟨*CloudXRViewer/Common/WindowStateManager.swift* 51⟩ +≡

```
// Handles window appearance event.
// Behavior: This will also dismiss all other existing windows.
//
// - Parameters:
//   - windowId: The identifier of the window that appeared
func windowOnAppear(windowId: String) {
    guard configured else {
        fatalError("WindowStateManager not initialized when windowOnAppear called")
    }

    Task { @MainActor in
        for winId in visibleWindows {
            if winId != windowId {
                dismissWindow(id: winId)
            }
        }
    }

    visibleWindows.insert(windowId)
}

// Handles window disappearance event.
//
// - Parameters:
//   - windowId: The identifier of the window that disappeared
func windowOnDisappear(windowId: String) {
    guard configured else {
        fatalError("WindowStateManager not initialized when windowOnDisappear called")
    }
    guard visibleWindows.contains(windowId) else {
        fatalError("windowOnDisappear called when window \(windowId) is already dismissed")
    }
```

```
    visibleWindows.remove(windowId)
}
```

Continued in chunks 54, 55, 56.

When the application becomes active with a paused session, the session is resumed. Only the classic immersive space is reopened here; the foveated session presents its own on resume.

54 ⟨*CloudXRViewer/Common/WindowStateManager.swift* 51⟩ +≡

```
// For cases that window is closed but onDisappear() won't trigger
func windowOnScenePhaseChange(scenePhase: ScenePhase) {
    guard configured else {
        fatalError("Window state manager not configured when scene phase changed")
    }
    if scenePhase == .inactive {
        for windowId in visibleWindows {
            dismissWindow(id: windowId)
        }
    } else if scenePhase == .active {
        if appModel.status == .paused {
            Task { @MainActor in
                try await appModel.activeSession?.resume()
                // The foveated session presents its own immersive space on resume.
                if appModel.backend == .classic && !immersiveSpaceActive {
                    await openImmersiveSpace(id: immersiveTitle)
                }
            }
        }
    }
}

func dismissImmersiveSpaceIfOpen() {
    if immersiveSpaceActive {
        Task { @MainActor in
            await dismissImmersiveSpace()
        }
    }
}
```

Continued in chunks 55, 56.

6.2 Following the connection

Connection changes arrive from the windows' observers (Chapter 2). A new connection switches to the content window and, for the classic backend, opens the immersive space; a disconnection switches back to the launch window and, again only for the classic backend, dismisses the space. For the foveated backend the session does both through its presentation behaviors, and doing it here as well would race with it.

55 ⟨*CloudXRViewer/Common/WindowStateManager.swift* 51⟩ +≡

```
func onConnectionStateChanged(oldState: ViewerStatus, newState: ViewerStatus) async {
    switch newState {
    case .connected:
        await handleConnectionEstablished()
    case .disconnected:
        await handleDisconnection()
    default:
        return
    }
}

private func switchWindow(newWindowId: String) {
    guard configured else {
```

```
            fatalError("Window manager not configured when switching window")
        }

        Task { @MainActor in
            if !visibleWindows.contains(newWindowId) {
                openWindow(id: newWindowId)
            }
        }
    }

    // Handles connection established event.
    // Behavior: Opens immersive space and optionally hides window based on settings.
    // The foveated session opens its immersive space itself (`immersivePresentationBehaviors`).
    private func handleConnectionEstablished() async {
        guard configured else {
            fatalError("Connection established before window manager was configured")
        }

        if appModel.backend == .classic && !immersiveSpaceActive {
            await openImmersiveSpace(id: immersiveTitle)
        }

        switchWindow(newWindowId: contentTitle)
    }

    // Handles disconnection event
    // Behavior: Restores window visibility and dismisses immersive space
    // (the foveated session dismisses its own).
    private func handleDisconnection() async {
        guard configured else {
            fatalError("Disconnection before window manager was configured")
        }

        switchWindow(newWindowId: launchTitle)
        if appModel.backend == .classic && immersiveSpaceActive == true {
            await dismissImmersiveSpace()
        }
    }
```

Continued in chunk 56.

6.3 Following the immersive space

The immersive views report when they appear and disappear. Pressing the Digital Crown dismisses the immersive space; the manager then pauses the active session, whichever backend it belongs to, rather than leaving a stream running with nothing to show it. For a foveated session the system pause sheet then offers to resume or disconnect. Finally, a triple tap in the immersive space toggles the windows.

56 ⟨*CloudXRViewer/Common/WindowStateManager.swift* 51⟩ +≡

```
    func immersiveSpaceOnAppear() {
        guard !immersiveSpaceActive else {
            fatalError("Immersive space was already active when onAppear was called")
        }

        immersiveSpaceActive = true
    }

    // Handles immersive space dismissal (especially triggered by crown press)
    // Behavior: Restores window visibility and pauses session if active
    func immersiveSpaceOnDisappear() {
        guard immersiveSpaceActive else {
```

```
            fatalError("Immersive space state incorrect on disappear")
        }

        guard configured else {
            fatalError("Window manager not configured when immersive space disappeared")
        }

        immersiveSpaceActive = false

        // If the launch window was not visible, switch to the launch window.
        switchWindow(newWindowId: launchTitle)

        // Pause the session if necessary
        switch appModel.status {
        case .disconnecting, .disconnected, .initialized, .pausing, .paused:
            return
        default:
            if pauseOnImmersiveSpaceDismissed {
                // If headset is removed, dismiss immersive space rather than leaving it in background.
                Task { @MainActor in
                    try? await appModel.activeSession?.pause()
                    if immersiveSpaceActive {
                        await dismissImmersiveSpace()
                    }
                }
            }
        }
    }

    // Toggles the visibility of the window based on the current immersive space.
    func toggleWindow() {
        guard configured else {
            fatalError("Window manager is not configured when toggling the window")
        }

        if !visibleWindows.isEmpty {
            for windowId in visibleWindows {
                dismissWindow(id: windowId)
            }
        }
        else {
            if immersiveSpaceActive {
                switchWindow(newWindowId: contentTitle)
            } else {
                switchWindow(newWindowId: launchTitle)
            }
        }
    }
}
#endif
```

The file ends without a newline.

6.4 The launch window

The launch view shows the form. Its completion handler opens the classic immersive space after a classic connection and, if the user chose so, hides the launch window. After a foveated connection the space is already being presented by the session.

57 ⟨*CloudXRViewer/visionOS/LaunchView+visionOS.swift* 57⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import RealityKit
import CloudXRKit

struct LaunchView: View {
    @Environment(\.openImmersiveSpace) var openImmersiveSpace
    @Environment(\.dismissImmersiveSpace) var dismissImmersiveSpace
    @Environment(\.openWindow) private var openWindow
    @Environment(\.dismissWindow) private var dismissWindow
    @Environment(\.scenePhase) var scenePhase
    @Environment(AppModel.self) var appModel


    @Binding var application: Application

    func showImmersiveSpace() {
        Task {
            // The foveated session presents its own immersive space when streaming starts.
            if appModel.backend == .classic {
                await openImmersiveSpace(id: immersiveTitle)
            }
            if !ViewerApp.persistLaunchWindow {
                dismissWindow(id: launchTitle)
            }
        }
    }

    var body: some View {

        if !application.isConfigurator {
                // Just show the launch view while streaming.
            VStack {
                Spacer(minLength: 24)
                SessionConfigView(application: $application) {
                    showImmersiveSpace()
                }
                Spacer(minLength: 24)
            }
            .glassBackgroundEffect()
            .onChange(of: scenePhase) {
                appModel.windowStateManager.windowOnScenePhaseChange(scenePhase: scenePhase)
            }
        }
    }
}

#Preview {
    @Previewable @State var appModel = AppModel()

    return LaunchView(application: $appModel.application)
        .environment(appModel)
}
```

Root chunk, tangled to `CloudXRViewer/visionOS/LaunchView+visionOS.swift`.

6.5 The configuration form

The visionOS layout of `SessionConfigView`. An ornament toggles NVIDIA's simplified view.

58 ⟨*CloudXRViewer/visionOS/SessionConfigView+visionOS.swift* 58⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import CloudXRKit
import Network

extension SessionConfigView {
    var body: some View {
        mainView
        .ornament(
            visibility: useSimpleConfigView ? .visible : .hidden,
            attachmentAnchor: .scene(.init(x: 0.98, y: -0.02)),
            contentAlignment: .bottomTrailing
        ) {
            Button {
                useSimpleConfigView.toggle()
            } label: {
                Image(systemName: "gearshape")
                    .padding(12)
            }
        }
    }
```

Continued in chunks 59, 60. Root chunk, tangled to `CloudXRViewer/visionOS/SessionConfigView+visionOS.swift`.

The form starts with the backend picker, shown only when the build offers more than one backend (Section 3.1) and disabled during a session. The classic section is NVIDIA's: zone, IP address or authentication method, hand tracking, application identifier and resolution preset.

59 ⟨*CloudXRViewer/visionOS/SessionConfigView+visionOS.swift* 58⟩ +≡

```
    @ViewBuilder
    var sessionConfigForm: some View {
        Form {
            if StreamingBackend.available.count > 1 {
                Section {
                    Picker("Streaming Backend", selection: backendSelection) {
                        ForEach(StreamingBackend.available, id: \.self) { Text($0.label) }
                    }
                    .pickerStyle(.segmented)
                    .disabled(sessionConnected)
                }
            }

            if appModel.backend == .classic {
                Section {
                    Picker("Select Zone", selection: $zone) {
                        ForEach (Zone.allCases, id: \.self) { Text($0.rawValue) }
                    }
                    .onChange(of: zone) {
                        if zone == .ipAddress {
                            // Dummy call to trigger request local network permissions early
                            NetServiceBrowser().searchForServices(ofType: "_http._tcp.", inDomain:
↪ "local.")
                        }
                    }

                    if zone == .ipAddress {
                        HStack {
                            Text("IP Address")
                            Spacer()
                            TextField("0.0.0.0", text: $hostAddress)
                                .autocapitalization(.none)
                                .multilineTextAlignment(.trailing)
```

```
                            .autocorrectionDisabled(true)
                            .textInputAutocapitalization(.never)
                            .keyboardType(.numbersAndPunctuation)
                            .searchDictationBehavior(.inline(activation: .onLook))
                            .onSubmit {
                                // strip whitespace
                                hostAddress = hostAddress.trimmingCharacters(in:
↪ .whitespacesAndNewlines)
                            }
                    }
                } else {
                    HStack {
                        Picker("Select Authentication Method", selection: $authMethod) {
                            ForEach (AuthMethod.allCases, id: \.self) { authOption in
                                Text(authOption.rawValue)
                            }
                        }
                    }
                }

                if !application.isConfigurator {
                    Toggle("Enable Hand Tracking", isOn: $enableHandTracking)
                }

                if zone != .ipAddress && application.appID == .unknown {
                    HStack {
                        Text("Enter Application ID")
                        Spacer()
                        TextField("", value: $genericAppID, format: .number.grouping(.never))
                            .disableAutocorrection(true)
                            .keyboardType(.numberPad)
                            .multilineTextAlignment(.trailing)
                    }
                }

                Section {
                    Picker("Resolution Preset", selection: $resolutionPreset) {
                        ForEach(ResolutionPreset.allCases, id: \.self) { preset in
                            Text(preset.rawValue)
                        }
                    }.disabled(sessionConnected)
                }
            }
        } else {
            foveatedConfigSection
        }
    }
    .frame(minHeight: 420, maxHeight: 790)
}
```

Continued in chunk 60.

The foveated section offers the endpoint kind and the fields it needs. Hand tracking is offered only on visionOS 27, where the input capability exists. Upstream sizes the form to a fixed height, so the explanation of foveation lives in the HUD tab instead of a footer here.

60 ⟨*CloudXRViewer/visionOS/SessionConfigView+visionOS.swift* 58⟩ +≡

```
@ViewBuilder
var foveatedConfigSection: some View {
    Section {
        Picker("Server", selection: $foveatedEndpointKind) {
            ForEach(FoveatedEndpointKind.allCases, id: \.self) { Text($0.rawValue) }
        }
        .onChange(of: foveatedEndpointKind) {
            if foveatedEndpointKind == .local {
```

```
                // Dummy call to trigger request local network permissions early
                NetServiceBrowser().searchForServices(ofType: "_http._tcp.", inDomain: "local.")
            }
        }

        switch foveatedEndpointKind {
        case .discover:
            EmptyView()
        case .local:
            HStack {
                Text("IP Address")
                Spacer()
                TextField("0.0.0.0", text: $hostAddress)
                    .multilineTextAlignment(.trailing)
                    .autocorrectionDisabled(true)
                    .textInputAutocapitalization(.never)
                    .keyboardType(.numbersAndPunctuation)
                    .onSubmit {
                        // strip whitespace
                        hostAddress = hostAddress.trimmingCharacters(in: .whitespacesAndNewlines)
                    }
            }
            HStack {
                Text("Port")
                Spacer()
                TextField("", value: $foveatedPort, format: .number.grouping(.never))
                    .keyboardType(.numberPad)
                    .multilineTextAlignment(.trailing)
            }
        case .remote:
            HStack {
                Text("Server Name")
                Spacer()
                TextField("server.example.com", text: $remoteServerName)
                    .multilineTextAlignment(.trailing)
                    .autocorrectionDisabled(true)
                    .textInputAutocapitalization(.never)
                    .keyboardType(.URL)
            }
        }

        if #available(visionOS 27.0, *) {
            Toggle("Enable Hand Tracking", isOn: $enableHandTracking)
        }
    }
    .disabled(sessionConnected)
  }
}
```

6.6 The content window

The content window hosts `GenericWindowUI`. Its ornament shows CloudXRKit's network quality and opens a statistics sheet; Foveated Streaming publishes no such statistics, so the ornament is hidden for that backend.

61 ⟨*CloudXRViewer/visionOS/MainContentView+visionOS.swift* 61⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import RealityKit
import CloudXRKit

struct MainContentView: View {
    @Environment(\.openImmersiveSpace) var openImmersiveSpace
    @Environment(\.dismissImmersiveSpace) var dismissImmersiveSpace
    @Environment(\.openWindow) private var openWindow
    @Environment(\.dismissWindow) private var dismissWindow
    @Environment(\.scenePhase) var scenePhase
    @Environment(AppModel.self) var appModel


    @Binding var application: Application
    @State var showLiveStats: Bool = false

    private func networkQualityIndicator(for quality: CloudXRKit.Session.SessionQuality?) -> (color:
    ↪ Color, value: Double) {
        switch quality {
        case .excellent:
            return (.green, 1.0)
        case .good:
            return (.green, 0.75)
        case .degraded:
            return (.yellow, 0.5)
        case .unsustainable:
            return (.red, 0.25)
        default:
            return (.gray, 0.0)
        }
    }

    func makeNetworkIcon() -> some View {
        let indicator = networkQualityIndicator(for: appModel.session?.sessionQuality)

        return Image(systemName: "cellularbars", variableValue: indicator.value)
            .foregroundStyle(indicator.color)
            .padding(12)
    }

    var body: some View {
        VStack {

            if !application.isConfigurator {
                GenericWindowUI(application: $application)
            }
        }
        .ornament(
            // CloudXRKit's network statistics; the foveated backend does not publish them.
            visibility: appModel.backend == .classic ? .visible : .hidden,
            attachmentAnchor: .scene(.init(x: 0.98, y: -0.02)),
            contentAlignment: .bottomTrailing
        ) {
            Button {
                showLiveStats.toggle()
            } label: {
                makeNetworkIcon()
            }
        }
        .sheet(isPresented: $showLiveStats) {
            NetworkStatsView(
```

```
                session: appModel.session,
                onClose: { showLiveStats = false }
            )
        }
    }
}
```

Continued in chunk 62. Root chunk, tangled to `CloudXRViewer/visionOS/MainContentView+visionOS.swift`.

The statistics sheet and its rows are NVIDIA's.

62 ⟨*CloudXRViewer/visionOS/MainContentView+visionOS.swift* 61⟩ +≡

```
struct NetworkStatsView: View {
    let session: Session?
    let onClose: () -> Void

    private var stats: (latency: String, jitter: String, bandwidth: String) {
        let latencyText = session?.liveNetworkStats.latencyMilliseconds.map(String.init) ?? "N/A"
        let jitterText = session?.liveNetworkStats.jitterMilliseconds.map(String.init) ?? "N/A"
        let bandwidthText = session?.liveNetworkStats.bandwidthMbps.map(String.init) ?? "N/A"
        return (latencyText, jitterText, bandwidthText)
    }

    var body: some View {
        VStack(spacing: 0) {
            // Header
            HStack {
                Text("Network Statistics")
                    .font(.title2)
                    .fontWeight(.semibold)

                Spacer()

                Button {
                    onClose()
                } label: {
                    Image(systemName: "xmark")
                        .padding(16)
                }
                .buttonStyle(.borderedProminent)
                .contentShape(Circle())
                .frame(width: 36, height: 36)
            }
            .padding(.horizontal, 36)
            .padding(.vertical, 24)

            Divider()
                .padding(.horizontal, 8) // Inset divider slightly

            // Content
            VStack(alignment: .leading, spacing: 18) {
                StatRow(title: "Latency", value: stats.latency, unit: "ms")
                StatRow(title: "Jitter", value: stats.jitter, unit: "ms")
                StatRow(title: "Bandwidth", value: stats.bandwidth, unit: "Mbps")
            }
            .padding(.horizontal, 36)
            .padding(.vertical, 24)
            .padding(.bottom, 4) // Extra bottom padding for rounded corners
        }
        .frame(minWidth: 320, maxWidth: 420)
        .fixedSize(horizontal: false, vertical: true)
        .background(.regularMaterial, in: RoundedRectangle(cornerRadius: 20))
        .presentationDetents([.height(220)]) // Slightly taller for better proportions
        .presentationDragIndicator(.visible)
        .presentationCornerRadius(20)
```

```
        .presentationBackground(.clear) // Let the background show through
    }
}

struct StatRow: View {
    let title: String
    let value: String
    let unit: String

    var body: some View {
        HStack(alignment: .center) {
            Text(title)
                .font(.body)
                .fontWeight(.medium)
                .foregroundStyle(.primary)
                .frame(maxWidth: .infinity, alignment: .leading)

            HStack(alignment: .firstTextBaseline, spacing: 6) {
                Text(value)
                    .font(.title3)
                    .fontWeight(.semibold)
                    .monospacedDigit()
                    .foregroundStyle(.primary)

                Text(unit)
                    .font(.caption)
                    .fontWeight(.medium)
                    .foregroundStyle(.secondary)
            }
            .frame(minWidth: 80, alignment: .trailing)
        }
        .padding(.vertical, 6)
        .padding(.horizontal, 4) // Small internal padding for better touch targets
    }
}

#Preview {
    @Previewable @State var appModel = AppModel()

    return MainContentView(application: $appModel.application)
        .environment(appModel)
}
```

6.7 Tabs of the content window

`GenericWindowUI` switches between three panels with buttons in an ornament. The selected channel is kept here, with the neutral types, so it survives switching tabs. The HUD panel shows CloudXRKit's HUD for the classic backend and a status view for the foveated one.

63 ⟨*CloudXRViewer/visionOS/WindowUI.swift* 63⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import CloudXRKit
import RealityKit

struct GenericWindowUI: View {
    enum Section {
        case hud
        case launch
```

```
        case actions

        var title: String {
            switch self {
            case .hud:
                "HUD"
            case .launch:
                "Config Screen"
            case .actions:
                "Actions"
            }
        }
    }

    @Environment(AppModel.self) var appModel
    @Environment(\.openImmersiveSpace) var openImmersiveSpace

    /// Current section being displayed
    @State var section = Section.launch

    // States for the selected opaque data channel.
    @State var currentChannelSelection: ViewerChannelID?
    @State var currentChannel: (any ViewerChannel)?

    @Binding var application: Application

    var body: some View {
#if DEBUG
        let _ = Self._printChanges()
#endif

        VStack {
            titleBar
                .padding(.all)

            // decide which panel to show in this window
            switch section {
            case .hud:
                if appModel.backend == .foveated {
                    FoveatedStatusView()
                } else if let session = appModel.session {
                    ScrollView {
                        HUDView(session: session, hudConfig: HUDConfig())
                    }
                }
            case .launch:
                SessionConfigView(application: $application) {}
            case .actions:
                ServerActionsView(
                    currentChannelSelection: $currentChannelSelection,
                    currentChannel: $currentChannel
                )
            }
        }
        .ornament(visibility: .visible, attachmentAnchor: .scene(.init(x: 0.5, y: 0.92))) {
            // view selection "tabs" along the bottom of the window
            HStack {
                Button("HUD") {
                    section = .hud
                }
                .selectedStyle(isSelected: section == .hud)
                Button("Config Screen") {
                    section = .launch
                }
```

```
                .selectedStyle(isSelected: section == .launch)
                Button("Server Actions") {
                    section = .actions
                }
                .selectedStyle(isSelected: section == .actions)
            }
            .ornamentStyle
        }
        // align all the useful information to the top of the window
        Spacer()
    }

    /// The titlebar at the top of the panel showing the panel name and controls at left and right
    var titleBar: some View {
        HStack {
            Spacer()

            VStack{
                // Title
                Text(section.title)
                    .font(UIConstants.titleFont)

                Button(appModel.ratingText) {
                    appModel.isRatingViewPresented = true
                }
                .disabled(appModel.disableFeedback)
                .sheet(isPresented: Binding(get: { appModel.isRatingViewPresented }, set: { _ in } )) {
                    StarRatingView()
                }
            }

            Spacer()
        }
    }
}
```

Continued in chunk 64. Root chunk, tangled to `CloudXRViewer/visionOS/WindowUI.swift`.

The foveated status view shows the backend, the endpoint kind and the session status, and explains what happens to the user's gaze.

64 ⟨*CloudXRViewer/visionOS/WindowUI.swift* 63⟩ +≡

```
/// The HUD tab for the foveated backend. Apple's framework does not publish streaming
/// statistics, so this shows the session state and what foveation does with the user's gaze.
struct FoveatedStatusView: View {
    @Environment(AppModel.self) var appModel
    @AppStorage("foveatedEndpointKind") var foveatedEndpointKind: FoveatedEndpointKind = .discover

    var body: some View {
        Form {
            LabeledContent("Backend", value: appModel.backend.label)
            LabeledContent("Server", value: foveatedEndpointKind.rawValue)
            LabeledContent("Status", value: appModel.activeSession?.statusDescription ?? "")
            Section {
                Text("Dynamic foveation is active: visionOS sends the approximate region you are looking
 ↪ at to the CloudXR runtime, which encodes that region at full quality and the periphery at lower
 ↪ quality. The streamed application never receives your gaze.")
                    .foregroundStyle(.secondary)
            }
        }
    }
}

#Preview {
    @Previewable @State var appModel = AppModel()
```

```
    return GenericWindowUI(application: $appModel.application)
        .environment(appModel)
}
```

The file ends without a newline.

6.8 The immersive views

The classic immersive view puts the CloudXRKit session on an entity, which CloudXRKit's systems render, and adds an invisible wall that receives taps. The wall is now a static function so the foveated view can reuse it.

65 ⟨*CloudXRViewer/visionOS/ImmersiveView.swift* 65⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import CloudXRKit
import SwiftUI
import RealityKit

struct GenericImmersiveView: View {
    @Environment(\.openWindow) private var openWindow
    @Environment(AppModel.self) var appModel

    @State private var sessionEntity = Entity()

    var body: some View {
        RealityView { content in
            sessionEntity.name = "Session"

            if let session = appModel.session {
                sessionEntity.components[CloudXRSessionComponent.self] = .init(session: session)
            }

            content.add(sessionEntity)

            content.add(Self.makeInvisibleGestureWall())
        } update: { content in

        }
    }

    static func makeInvisibleGestureWall() -> Entity {
        // Add an invisible plane that covers the viewport, attached to the headset that can accept
    ↪ gestures
        // so as to not get in the way of gestures on UI objects, the plane is 20 meters away.
        let plane = Entity()
        plane.components.set(InputTargetComponent())
        var collision = CollisionComponent(shapes: [.generateBox(width: 40, height: 40, depth: 0.01)])
        collision.mode = .trigger
        plane.components.set(collision)
        plane.position.z = -20
        return plane
    }
}
```

The file ends without a newline. Root chunk, tangled to `CloudXRViewer/visionOS/ImmersiveView.swift`.

The foveated immersive view has no session entity: the system draws the stream in the space. It adds the same gesture wall and reports the same lifecycle events as the classic view, so the window manager treats both spaces alike.

66 ⟨*CloudXRViewer/visionOS/FoveatedImmersiveView.swift* 66⟩ ≡
⟨*MIT permission notice* 1⟩

```
// Copyright (c) 2026 Pablo Figueroa and Claude.

import SwiftUI
import RealityKit

/// Overlay content for the foveated streaming immersive space.
///
/// `ImmersiveSpace(foveatedStreaming:)` draws the stream itself, so unlike `GenericImmersiveView`
/// there is no session entity here: only the gesture wall that lets a triple tap bring back the
/// window, and the same lifecycle hooks the classic immersive space uses.
struct FoveatedImmersiveView: View {
    @Environment(AppModel.self) var appModel
    @Environment(\.scenePhase) var scenePhase

    var body: some View {
        RealityView { content in
            content.add(GenericImmersiveView.makeInvisibleGestureWall())
        }
        .onAppear {
            appModel.windowStateManager.immersiveSpaceOnAppear()
        }
        .onDisappear {
            appModel.windowStateManager.immersiveSpaceOnDisappear()
        }
        .onTapGesture(count: 3) {
            appModel.windowStateManager.toggleWindow()
        }
        .onChange(of: scenePhase) {
            if scenePhase == .background { // Handle headset doffing during session
                appModel.windowStateManager.dismissImmersiveSpaceIfOpen()
            }
        }
    }
}
```

Root chunk, tangled to `CloudXRViewer/visionOS/FoveatedImmersiveView.swift`.

6.9 Styles and placement helpers

The ornament style used by the tab bar.

67 ⟨*CloudXRViewer/visionOS/UIStyles+visionOS.swift* 67⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI

struct OrnamentStyle: ViewModifier {
    func body(content: Content) -> some View {
        content
            .toggleStyle(.button)
            .buttonStyle(.borderless)
            .labelStyle(.iconOnly)
            .padding(12)
            .glassBackgroundEffect(in: .rect(cornerRadius: 50))
    }
}

extension View {
    /// allows adding `.ornamentStyle` to any view to use `OrnamentStyle` on it
    var ornamentStyle: some View {
        modifier(OrnamentStyle())
```

```
    }
}
```

The file ends without a newline. Root chunk, tangled to `CloudXRViewer/visionOS/UIStyles+visionOS.swift`.

Small vector, matrix and quaternion conveniences from NVIDIA's code.

68 ⟨*CloudXRViewer/visionOS/Placement/Placement+Extensions.swift* 68⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import simd
import Spatial
import RealityKit

public extension simd_float3 {
    static var xAxis: simd_float3 { [1, 0, 0] }
    static var yAxis: simd_float3 { [0, 1, 0] }
    static var zAxis: simd_float3 { [0, 0, 1] }
}

public extension simd_float4x4 {
    static var identity: simd_float4x4 { matrix_identity_float4x4 }
    static var zero: simd_float4x4 { simd_float4x4() }

    init(radians: Float, axis: simd_float3) {
        let quat = simd_quatf(angle: radians, axis: axis)
        self.init(quat)
    }

    static func translation(_ x: Float, _ y: Float, _ z: Float) -> simd_float4x4 {
        var matrix: simd_float4x4 = .identity
        matrix[3] = [x, y, z, 1.0]
        return matrix
    }
}

public extension simd_quatf {
    static var identity = simd_quatf.init(matrix_float4x4.identity)
}
```

Root chunk, tangled to `CloudXRViewer/visionOS/Placement/Placement+Extensions.swift`.

Chapter 7

The iOS User Interface

On iOS the whole application lives in one window. The configuration form fills it until a session connects; then the stream fills it, with buttons along the bottom edge. The foveated backend here is Apple's StreamingSession package, which mirrors Foveated Streaming's API but has no eye tracking.

7.1 The launch view

The launch view chooses between the streaming view and the form. Two modifiers from the StreamingSession package attach the system's user interface for finding servers and for scanning a server's pairing QR code; they present it when a session connects with system discovery.

69 ⟨*CloudXRViewer/iOS/LaunchView+iOS.swift* 69⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import RealityKit
import CloudXRKit
import StreamingSession

struct LaunchView: View {
    @Environment(AppModel.self) var appModel
    @Environment(\.colorScheme) var colorScheme


    @Binding var application: Application

    var body: some View {

        if !application.isConfigurator {
            Group {
                if appModel.showStreamingAppView {
                    BasicStreamingView(application: $application)
                } else {
                    VStack {
                        Spacer(minLength: 24)
                        SessionConfigView(application: $application) {}
                        Spacer(minLength: 24)
                    }
                }
            }
            // System UI for finding servers and scanning their pairing QR code.
            .deviceDiscovery(session: appModel.foveatedSession)
            .devicePairing(session: appModel.foveatedSession)
        }
    }
```

```
}

#Preview {
    @Previewable @State var appModel = AppModel()

    appModel.application = .generic_viewer
    appModel.session = CloudXRSession(config: Config())

    return LaunchView(application: $appModel.application)
        .environment(appModel)
}
```

The file ends without a newline. Root chunk, tangled to `CloudXRViewer/iOS/LaunchView+iOS.swift`.

7.2 The configuration form

The iOS layout of `SessionConfigView`, with the same structure as on visionOS and fixed-width rows.

70 ⟨*CloudXRViewer/iOS/SessionConfigView+iOS.swift* 70⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import CloudXRKit
import Network

extension SessionConfigView {
    var body: some View {
        mainView
    }

    @ViewBuilder
    var sessionConfigForm: some View {
        Form {
            if StreamingBackend.available.count > 1 {
                Section {
                    Picker("Streaming Backend", selection: backendSelection) {
                        ForEach(StreamingBackend.available, id: \.self) { Text($0.label) }
                    }
                    .pickerStyle(.segmented)
                    .frame(width: 500)
                    .disabled(sessionConnected)
                }
            }

            if appModel.backend == .classic {
                Section {
                    VStack {
                        Picker("Select Zone", selection: $zone) {
                            ForEach (Zone.allCases, id: \.self) { Text($0.rawValue) }
                        }
                        .frame(width: 500)
                        .onChange(of: zone) {
                            if zone == .ipAddress {
                                // Dummy call to trigger request local network permissions early
                                NetServiceBrowser().searchForServices(ofType: "_http._tcp.", inDomain:
    ↪ "local.")
                            }
                        }

                        if zone == .ipAddress {
                            HStack {
                                Text("IP Address")
```

```
                                // TODO(chaoyehc): Investigate why removing the padding
                                // causes slight misalignment in the text.
                                .padding(.vertical, 7)
                                .frame(alignment: .leading)
                            Spacer()
                            TextField("0.0.0.0", text: $hostAddress)
                                .autocapitalization(.none)
                                .multilineTextAlignment(.trailing)
                                .autocorrectionDisabled(true)
                                .textInputAutocapitalization(.never)
                                .keyboardType(.numbersAndPunctuation)
                                .searchDictationBehavior(.inline(activation: .onLook))
                                .onSubmit {
                                    // strip whitespace
                                    hostAddress = hostAddress.trimmingCharacters(in:
↪ .whitespacesAndNewlines)
                                }
                        }
                        .frame(width: 500)
                    } else {
                        HStack {
                            Picker("Select Authentication Method", selection: $authMethod) {
                                ForEach (AuthMethod.allCases, id: \.self) { authOption in
                                    Text(authOption.rawValue)
                                }
                            }
                            .frame(width: 500)
                        }
                    }

                    HStack {
                        Text("Resolution Preset")
                            .frame(alignment: .leading)
                        Spacer()
                        Picker("", selection: $resolutionPreset) {
                            ForEach(ResolutionPreset.allCases, id: \.self) { preset in
                                Text(preset.rawValue)
                            }
                        }
                        .disabled(sessionConnected)
                    }
                    .frame(width: 500)

                    if zone != .ipAddress && application.appID == .unknown {
                        HStack {
                            Text("Enter Application ID")
                                .frame(alignment: .leading)
                            Spacer()
                            TextField("", value: $genericAppID, format: .number.grouping(.never))
                                .disableAutocorrection(true)
                                .keyboardType(.numberPad)
                                .multilineTextAlignment(.trailing)
                        }
                        .frame(width: 500)
                    }
                }
            }
        } else {
            foveatedConfigSection
        }
    }
    .padding(.top, 100)
    .frame(minWidth: 600, maxWidth: 600, minHeight: 360, maxHeight: 690)
}
```

Continued in chunk 71. Root chunk, tangled to `CloudXRViewer/iOS/SessionConfigView+iOS.swift`.

The foveated section adds a note, since on iOS the backend streams without foveation.

71 ⟨ *CloudXRViewer/iOS/SessionConfigView+iOS.swift* 70⟩ +≡

```
/// Shown under the foveated streaming settings.
var foveationNote: String {
    "Streams with Apple's StreamingSession framework. Foveation needs eye tracking, which is only
    ↪ available on Apple Vision Pro."
}

@ViewBuilder
var foveatedConfigSection: some View {
    Section {
        VStack {
            Picker("Server", selection: $foveatedEndpointKind) {
                ForEach(FoveatedEndpointKind.allCases, id: \.self) { Text($0.rawValue) }
            }
            .frame(width: 500)
            .onChange(of: foveatedEndpointKind) {
                if foveatedEndpointKind == .local {
                    // Dummy call to trigger request local network permissions early
                    NetServiceBrowser().searchForServices(ofType: "_http._tcp.", inDomain: "local.")
                }
            }

            switch foveatedEndpointKind {
            case .discover:
                EmptyView()
            case .local:
                HStack {
                    Text("IP Address")
                    Spacer()
                    TextField("0.0.0.0", text: $hostAddress)
                        .multilineTextAlignment(.trailing)
                        .autocorrectionDisabled(true)
                        .textInputAutocapitalization(.never)
                        .keyboardType(.numbersAndPunctuation)
                        .onSubmit {
                            // strip whitespace
                            hostAddress = hostAddress.trimmingCharacters(in: .whitespacesAndNewlines)
                        }
                }
                .frame(width: 500)
                HStack {
                    Text("Port")
                    Spacer()
                    TextField("", value: $foveatedPort, format: .number.grouping(.never))
                        .keyboardType(.numberPad)
                        .multilineTextAlignment(.trailing)
                }
                .frame(width: 500)
            case .remote:
                HStack {
                    Text("Server Name")
                    Spacer()
                    TextField("server.example.com", text: $remoteServerName)
                        .multilineTextAlignment(.trailing)
                        .autocorrectionDisabled(true)
                        .textInputAutocapitalization(.never)
                        .keyboardType(.URL)
                }
                .frame(width: 500)
            }
```

```
                Text(foveationNote)
                    .font(.footnote)
                    .foregroundStyle(.secondary)
                    .frame(width: 500, alignment: .leading)
            }
            .disabled(sessionConnected)
        }
    }
}
```

7.3 The streaming view

The stream is drawn by `FoveatedStreamingView` for the StreamingSession backend and by a RealityKit view with the CloudXRKit session component for the classic one. The HUD and its button exist only for the classic backend.

72 ⟨*CloudXRViewer/iOS/StreamingView+iOS.swift* 72⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import RealityKit
import CloudXRKit
import StreamingSession
import ARKit
import Foundation

struct BasicStreamingView : View {
    enum Section {
        case none
        case launch
        case actions
    }

    @Environment(AppModel.self) var appModel

    @Environment(\.colorScheme) var colorScheme

    @State private var sessionEntity = Entity()
    @State var showHUD = false

    @State var selectedSection: Section = .none

    @Binding var application: Application

    @State var currentChannelSelection: ViewerChannelID?
    @State var currentChannel: (any ViewerChannel)?

    var body: some View {
        ZStack(alignment: .bottom) {
            if appModel.backend == .foveated {
                FoveatedStreamingView(session: appModel.foveatedSession)
                    .edgesIgnoringSafeArea(.all)
            } else {
                classicStreamView
            }

            if showHUD && appModel.backend == .classic {
                hudView
            }
```

```
            switch selectedSection {
            case .launch:
                SessionConfigView(application: $application) {}
            case .none:
                EmptyView()
            case .actions:
                ServerActionsView(
                    currentChannelSelection: $currentChannelSelection,
                    currentChannel: $currentChannel
                )
                    .frame(minHeight: 160, maxHeight: 690)
            }

            // Show the overylay UI only when connected.
            if appModel.status == .connected {
                VStack {
                    Spacer()
                    // Button to enable/disable HUD.
                    HStack {
                        actionsButton
                        Spacer()
                        configButton
                        if appModel.backend == .classic {
                            Spacer()
                            hudButton
                        }
                    }
                }
                .padding([.leading, .trailing, .bottom], 16)
            } else {
                // Show black/white background instead of passthrough when not connected.
                (colorScheme == .dark ? Color.black : Color.white).ignoresSafeArea()
                // Show the SessionConfigView for user to reconnect.
                SessionConfigView(application: $application) {}
            }
        }
    }
```

Continued in chunk 73. Root chunk, tangled to `CloudXRViewer/iOS/StreamingView+iOS.swift`.

The classic stream view uses ARKit's spatial tracking camera, so the stream is composited over the camera image.

73 ⟨*CloudXRViewer/iOS/StreamingView+iOS.swift* 72⟩ +≡

```
    /// CloudXRKit renders through a RealityKit entity carrying the session component.
    private var classicStreamView: some View {
        RealityView { content in
            sessionEntity.name = "Session"
            // The camera that displays virtual RealityKit content and camera passthrough, with tracking
    ↪ capabilities.
            content.camera = .spatialTracking
            if let session = appModel.session {
                sessionEntity.components[CloudXRSessionComponent.self] = .init(session: session)
            }

            content.add(sessionEntity)
        }
        .edgesIgnoringSafeArea(.all)
    }
}

#Preview {
    @Previewable @State var appModel = AppModel()

    return BasicStreamingView(application: $appModel.application)
```

```
        .environment(appModel)
}
```

The file ends without a newline.

7.4 Overlay buttons

The buttons along the bottom edge and the HUD overlay, unchanged from NVIDIA's code.

74 ⟨*CloudXRViewer/iOS/StreamingView+extensions+iOS.swift* 74⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import CloudXRKit
import SwiftUI
import RealityKit

extension BasicStreamingView {
    private var configButtonImage: String {
        switch selectedSection {
        case .launch:
            "gearshape.fill"
        default:
            "gearshape"
        }
    }

    internal var configButton: some View {
        customButton(
            image: configButtonImage,
            text: "Settings",
            action: {
                if selectedSection == .none {
                    selectedSection = .launch
                } else {
                    selectedSection = .none
                }
            },
            isDisabled: selectedSection != .none && selectedSection != .launch
        )
    }

    private var hudImage: String {
        showHUD ? "chart.bar.fill" : "chart.bar"
    }

    internal var hudButton: some View {
        customButton(
            image: hudImage,
            text: "Statistics",
            action: {
                showHUD.toggle()
            }
        )
    }

    private var actionsImage: String {
        selectedSection == .actions ? "hand.draw.fill" : "hand.draw"
    }

    internal var actionsButton: some View {
        customButton(
            image: actionsImage,
            text: "Actions",
```

```
            action: {
                if selectedSection == .none {
                    selectedSection = .actions
                } else {
                    selectedSection = .none
                }
            },
            isDisabled: selectedSection != .none && selectedSection != .actions
        )
    }

    internal var hudView: some View {
        HStack{
            Spacer()
            VStack {
                if let session = appModel.session {
                    HUDView(session: session, hudConfig: HUDConfig())
                        .frame(minWidth: 200, minHeight: 200)
                        .background(Color.black.opacity(0.7))
                        .cornerRadius(12)
                        .padding(.top, 50)
                }
                Spacer()
            }
            .padding(.top, 20)
        }
        .padding(.trailing, 20)
    }
}
```

The file ends without a newline. Root chunk, tangled to `CloudXRViewer/iOS/StreamingView+extensions+iOS.swift`.

Chapter 8

Opaque Data Channels

Both libraries let client and server exchange application messages over named channels, which the server opens. NVIDIA's viewer has a panel to try them: pick a channel, send "Action 1" or "Action 2", and watch the replies. The panel now uses the neutral channel types of Chapter 3, so it works with the LÖVR sample server NVIDIA provides and with the message channel in Apple's OpenXR sample alike.

8.1 Sending

The selected channel is a binding, owned by the window that hosts the panel. Sending encodes the text as UTF-8 and reports either the message or the error.

75 ⟨*CloudXRViewer/Common/ServerActionsView.swift* 75⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import os.log

struct ServerActionsView: View {
    static var logger = Logger()
    // Messages are sent via MessageChannel of the session in appModel.
    @Environment(AppModel.self) var appModel

    @State var lastMessageSent: String = ""
    @State var lastMessageReceived: String = ""

    @Binding var currentChannelSelection: ViewerChannelID?
    @Binding var currentChannel: (any ViewerChannel)?

    // Reader task for the currently selected channel.
    @State private var channelReaderTask: Task<Void, Never>?
    @State private var receivedMessageCount: Int = 0

    func sendMessage(message: String) {
        guard let channelSelection = currentChannelSelection else {
            Self.logger.warning("No channel selected")
            lastMessageSent = "Error - no channel"
            return
        }

        guard let messageData = message.data(using: .utf8) else {
            Self.logger.warning("String message could not be converted to data")
            lastMessageSent = "Error"
            return
        }
```

```
        if let channel = currentChannel {
            do {
                try channel.send(messageData)
                lastMessageSent = message
            } catch {
                Self.logger.warning("Failed to send message via current channel:
↪ \(error.localizedDescription)")
                lastMessageSent = "Error - failed to send"
            }
        } else {
            Self.logger.warning("No current channel available for send")
            lastMessageSent = "Error - no channel"
        }
    }
```

Continued in chunk 76. Root chunk, tangled to `CloudXRViewer/Common/ServerActionsView.swift`.

8.2 The panel

The picker lists the active session's channels. Selecting one starts a task that reads the channel's message stream and shows each message with a counter; selecting another, or the channel disappearing, cancels the task. The reader restarts when the panel reappears.

76 ⟨*CloudXRViewer/Common/ServerActionsView.swift* 75⟩ +≡

```
    var body: some View {
        Form {
            VStack {
                if let session = appModel.activeSession {
                    Picker("Channels", selection: $currentChannelSelection) {
                        ForEach(session.channelIDs, id: \.self) { channelID in
                            Text(channelID.label).tag(channelID as ViewerChannelID?)
                        }
                        Text("None").tag(nil as ViewerChannelID?)
                    }
                    .pickerStyle(.menu)
                    .id(session.channelIDs)
                    .onChange(of: currentChannelSelection) {
                        currentChannel = nil
                        // Cancel any existing reader task when switching selection.
                        channelReaderTask?.cancel()

                        guard let channelSelection = currentChannelSelection else {
                            return
                        }
                        guard let channel = session.channel(for: channelSelection) else {
                            return
                        }

                        currentChannel = channel
                        // Start a reader task for the selected channel.
                        channelReaderTask = Task {
                            for await message in channel.receivedMessageStream {
                                receivedMessageCount += 1
                                lastMessageReceived = "Message \(receivedMessageCount): " +
↪ String(decoding: message, as: UTF8.self)
                            }
                        }
                    }
                    .onChange(of: session.channelIDs) {
                        if let channelSelection = currentChannelSelection,
                           !session.channelIDs.contains(channelSelection)
                        {
```

```
                    currentChannelSelection = nil
                    // Cancel reader if selection disappears.
                    channelReaderTask?.cancel()
                    channelReaderTask = nil
                }
            }

            if let channel = currentChannel {
                Text("Status: \(channel.statusDescription)")
            } else {
                Text("Status: N/A")
            }
        }

        Divider()

        VStack {
            HStack {
                Spacer()
                Button("Action 1") {
                    sendMessage(message: "Action 1")
                }
                .disabled(currentChannelSelection == nil)
                .buttonStyle(.borderedProminent)
                Spacer()
                Button("Action 2") {
                    sendMessage(message: "Action 2")
                }
                .disabled(currentChannelSelection == nil)
                .buttonStyle(.borderedProminent)
                Spacer()
            }
        }
        Divider()
        VStack {
            Text("Last message sent: ")
            Divider()
            Text(lastMessageSent)
            Spacer()
        }
        Divider()
        VStack {
            Text("Last message received:")
            Divider()
            Text(lastMessageReceived)
            Spacer()
        }
    }
}
.onAppear {
    if let channel = currentChannel {
        channelReaderTask?.cancel()
        channelReaderTask = Task {
            for await message in channel.receivedMessageStream {
                receivedMessageCount += 1
                lastMessageReceived = "Message \(receivedMessageCount): " + String(decoding:
↪ message, as: UTF8.self)
            }
        }
    }
}
.onDisappear {
    channelReaderTask?.cancel()
    channelReaderTask = nil
```

```
        }
    }
}

#Preview {
    @Previewable @State var appModel = AppModel()
    @Previewable @State var selection: ViewerChannelID? = nil
    @Previewable @State var channel: (any ViewerChannel)? = nil
    ServerActionsView(currentChannelSelection: $selection, currentChannel: $channel)
        .environment(appModel)
}
```

The file ends without a newline.

Chapter 9

Remaining Pieces

This chapter collects the configuration form's helpers and the small utilities the views use.

9.1 Form helpers

NVIDIA's enumerations for the authentication method, the GDN zone, application identifiers and applications.

77 ⟨*CloudXRViewer/Common/SessionConfigView+extensions.swift* 77⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import CloudXRKit
import OSLog

enum AuthMethod: String, CaseIterable {
    case starfleet = "Geforce NOW login"
    case guest = "Guest Mode"
}

enum Zone: String, CaseIterable {
    case auto = "Auto"
    case us_east = "US East"
    case us_northwest = "US Northwest"
    case us_west = "US West"
    case eu_north = "EU North"
    case ipAddress = "Manual IP address"

    var id: String? {
        switch self {
        case .auto:
            nil // automatic
        case .us_east:
           "np-atl-03" // "us-east"
        case .us_northwest:
           "np-pdx-01" // "us-northwest"
        case .us_west:
            "np-sjc6-04" // "us-west"
        case .eu_north:
            "np-sth-04" // "eu-north"
        default:
            nil
        }
    }
}

enum AppID: UInt, CaseIterable {
```

```
    // Add CMS IDs here
    case unknown

    var rawValue: UInt {
        switch self {
        case .unknown:
            return 000_000_000
        }
    }
}

enum Application: String, CaseIterable {


    case generic_viewer = "Generic Viewer"

    var appID: AppID {
        switch self {
            // Add mapping from Applications to CMS IDs here
        default:
            .unknown
        }
    }

    var isConfigurator: Bool {
        switch self {
        case .generic_viewer:
            false

        }
    }
}
```

Continued in chunk 78. Root chunk, tangled to `CloudXRViewer/Common/SessionConfigView+extensions.swift`.

The form's computed properties. The backend binding writes through to the model. The status line shows the active session's own description, followed by the last foveated connection error if there is one. The button label and its enabled state follow the neutral status.

78 ⟨*CloudXRViewer/Common/SessionConfigView+extensions.swift* 77⟩ +≡

```
extension SessionConfigView {

    var backendSelection: Binding<StreamingBackend> {
        Binding(
            get: { appModel.backend },
            set: { appModel.backend = $0 }
        )
    }

    var stateDescription: String {
        let description = appModel.activeSession?.statusDescription ?? ""
        if appModel.backend == .foveated, let error = appModel.foveatedConnectError {
            return "\(description): \(error)"
        }
        return description
    }

    var buttonLabel: String {
        switch appModel.status {
        case .connected: "Disconnect"
        case .paused, .pausing: "Resume"
        default: "Connect"
        }
    }
```

```
    var usingGuestMode: Bool {
        authMethod == .guest && zone != .ipAddress
    }

    var connectButtonDisabled: Bool {
        switch appModel.status {
        case .connecting, .disconnecting, .resuming, .pausing:
            true
        default:
            false
        }
    }
}
```

The file ends without a newline.

9.2 Feedback

The rating sheet sends feedback through CloudXRKit's session.

79 ⟨*CloudXRViewer/Common/StarRatingView.swift* 79⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI
import CloudXRKit
import ARKit

struct StarRatingView: View {

    @State private var selectedReason = FeedbackReason.none
    @State private var rating: Int = 0
    @State private var ratingSelected = false

    @Environment(AppModel.self) var appModel
    var body: some View {
        VStack {
            Text("Rate your experience")
                .font(.title)
                .padding()
            HStack {
                ForEach(1...5, id: \.self) { star in
                    Image(systemName: star <= rating ? "star.fill" : "star")
                        .foregroundColor(star <= rating ? .yellow : .gray)
                        .onTapGesture {
                            rating = star
                            ratingSelected = true
                            // 5-star rating does not require a reason
                            if rating == 5 {
                                selectedReason = FeedbackReason.none
                            }
                        }
                }
            }
            .font(.largeTitle)
            .padding()

            if ratingSelected && rating < 5 {
                Picker("Select a reason", selection: $selectedReason) {
                    ForEach(FeedbackReason.allCases, id: \.self) {
                        if $0 == .none {
                            Text("Please select a reason")
                        } else {
                            Text($0.rawValue)
```

```
                    }
                }
            }
            .pickerStyle(MenuPickerStyle())
            .padding()
        }

        Button("Send feedback") {
            appModel.disableFeedback = true
            if selectedReason == FeedbackReason.none {
                appModel.ratingText = "Score: \(rating)"
            } else {
                appModel.ratingText = "Score: \(rating), Reason: \(selectedReason.rawValue)"
            }
            appModel.isRatingViewPresented = false
            appModel.session?.sendUserFeedback(rating: rating, selectedReason: selectedReason)
        }
        .disabled(rating==0)
        .padding()
    }
  }
}
```

The file ends without a newline. Root chunk, tangled to `CloudXRViewer/Common/StarRatingView.swift`.

9.3 Styles and constants

Layout constants and fonts.

80 ⟨*CloudXRViewer/Common/UIConstants.swift* 80⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI

struct UIConstants {
    /// general margin
    static let margin: CGFloat = 20

    /// size of image assets (trim is subject to being 2/3 this size)
    static let assetWidth: CGFloat = 277

    /// size of font used in toolbar at the bottom of the view
    static let toolbarFont: Font = .custom("SF Pro", size: 17)
        .leading(.loose)
        .weight(.bold)

    /// size of font used to headline each section of the window
    static let sectionFont: Font = .custom("SF Pro", size: 24)
        .leading(.loose)
        .weight(.bold)

    /// size of font used in the titlebar of the window
    static let titleFont: Font = .custom("SF Pro", size: 29)
        .leading(.loose)
        .weight(.bold)
}
```

The file ends without a newline. Root chunk, tangled to `CloudXRViewer/Common/UIConstants.swift`.

The selected-tab style and the overlay button used on iOS.

81 ⟨*CloudXRViewer/Common/UIStyles.swift* 81⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI

struct SelectedStyle: ViewModifier {
    var isSelected: Bool
    func body(content: Content) -> some View {
        if isSelected {
            ZStack {
                content
                    .buttonStyle(.borderedProminent)
                    .tint(Color(white: 1.0, opacity: 0.1))
            }
        } else {
            content
        }
    }
}

extension View {
    /// allows adding `.coatSelectedStyle(isSelected: true)` to make the view look "selected" per
    ↪ `COATSelectedStyle`
    func selectedStyle(isSelected: Bool) -> some View {
        modifier(SelectedStyle(isSelected: isSelected))
    }
}

struct CustomButtonStyle: ButtonStyle {
    let faint = Color(red: 1, green: 1, blue: 1, opacity: 0.05)
    var isDisabled = false
    func makeBody(configuration: Self.Configuration) -> some View {
        if isDisabled {
            configuration.label
                .background(.clear)
        } else {
            configuration.label
                .background(configuration.isPressed ? faint : .clear)
                .hoverEffect(.lift)
        }
    }
}

func customButton(image: String, text: String, action: @escaping () -> Void, isDisabled: Bool = false) -
    ↪ > some View {
    Button(
        action: action,
        label: {
            ZStack(alignment: .leading) {
                RoundedRectangle(cornerRadius: 15)
                    .fill(Color.black.opacity(0.6))
                    .frame(width: 170, height: 60)
                HStack {
                    Image(systemName: image)
                        .font(.largeTitle)
                        .foregroundColor(.white)

                    Text(text)
                        .font(.body)
                        .foregroundColor(.white)
                }
                .padding(.leading, 20)
            }
        }
```

```
    )
    .disabled(isDisabled)
    .opacity(isDisabled ? 0.5 : 1)
}
```

The file ends without a newline. Root chunk, tangled to `CloudXRViewer/Common/UIStyles.swift`.

A color from a hexadecimal integer.

82 ⟨*CloudXRViewer/Common/Color+hex.swift* 82⟩ ≡
⟨*NVIDIA license header* 2⟩

```
import SwiftUI

extension Color {
    /// initialize colour using hex integer
    init(hex: UInt, alpha: Double = 1) {
        self.init(
            .sRGB,
            red: Double((hex >> 16) & 0xff) / 255.0,
            green: Double((hex >> 08) & 0xff) / 255.0,
            blue: Double((hex >> 00) & 0xff) / 255.0,
            opacity: alpha
        )
    }
}
```

The file ends without a newline. Root chunk, tangled to `CloudXRViewer/Common/Color+hex.swift`.

9.4 Debug printing

`dprint` prints only in debug builds.

83 ⟨*CloudXRViewer/Common/Utilities.swift* 83⟩ ≡
⟨*NVIDIA license header* 2⟩

```
#if DEBUG
func dprint(_ strings: String...) {
    print(strings.joined(separator: " "))
}
#else
func dprint(_ str: String...) { }
#endif
```

Root chunk, tangled to `CloudXRViewer/Common/Utilities.swift`.

Bibliography

[1] Donald E. Knuth. Literate Programming. *The Computer Journal* 27(2):97–111, 1984.

[2] Norman Ramsey. Literate Programming Simplified. *IEEE Software* 11(5):97–105, 1994.

[3] NVIDIA. CloudXR Generic Viewer for Apple platforms. `https://github.com/NVIDIA/cloudxr-apple-generic-viewer`.

[4] NVIDIA. CloudXR SDK documentation: Foveated Streaming Framework. `https://docs.nvidia.com/cloudxr-sdk/latest/usr_guide/foveated_streaming/index.html`.

[5] Apple. Foveated Streaming framework documentation. `https://developer.apple.com/documentation/foveatedstreaming`.

[6] Apple. StreamingSession: streaming immersive content from a CloudXR application to visionOS and iOS. `https://github.com/apple/streamingsession`.

[7] Brian Guenter, Mark Finch, Steven Drucker, Desney Tan and John Snyder. Foveated 3D Graphics. *ACM Transactions on Graphics (SIGGRAPH Asia)* 31(6):164, 2012.

Index of Chunks

Every chunk name, the chunks that define it, and the chunks that use it. Root chunks are tangled to the file named by the chunk.

⟨*CloudXRViewer/Common/AppDelegate.swift* 19⟩
Defined in 19. Used in root.

⟨*CloudXRViewer/Common/AppModel.swift* 13⟩
Defined in 13, 14, 15, 16, 17, 18. Used in root.

⟨*CloudXRViewer/Common/Color+hex.swift* 82⟩
Defined in 82. Used in root.

⟨*CloudXRViewer/Common/GuestMode.swift* 39⟩
Defined in 39. Used in root.

⟨*CloudXRViewer/Common/ServerActionsView.swift* 75⟩
Defined in 75, 76. Used in root.

⟨*CloudXRViewer/Common/SessionConfigView+extensions.swift* 77⟩
Defined in 77, 78. Used in root.

⟨*CloudXRViewer/Common/SessionConfigView.swift* 32⟩
Defined in 32, 33, 34, 35, 36, 37, 38. Used in root.

⟨*CloudXRViewer/Common/StarRatingView.swift* 79⟩
Defined in 79. Used in root.

⟨*CloudXRViewer/Common/Streaming/ClassicSession.swift* 28⟩
Defined in 28, 29, 30, 31. Used in root.

⟨*CloudXRViewer/Common/Streaming/FoveatedSession.swift* 40⟩
Defined in 40, 41, 42, 43, 44, 45. Used in root.

⟨*CloudXRViewer/Common/Streaming/ViewerSession.swift* 21⟩
Defined in 21, 22, 23, 24, 25, 26, 27. Used in root.

⟨*CloudXRViewer/Common/TestHelper.swift* 20⟩
Defined in 20. Used in root.

⟨*CloudXRViewer/Common/UIConstants.swift* 80⟩
Defined in 80. Used in root.

⟨*CloudXRViewer/Common/UIStyles.swift* 81⟩
Defined in 81. Used in root.

⟨*CloudXRViewer/Common/Utilities.swift* 83⟩
Defined in 83. Used in root.

⟨*CloudXRViewer/Common/ViewerApp.swift* 3⟩
Defined in 3, 4, 5, 6, 7, 8, 9, 10, 11, 12. Used in root.

⟨*CloudXRViewer/Common/WindowStateManager.swift* 51⟩
Defined in 51, 52, 53, 54, 55, 56. Used in root.

⟨*CloudXRViewer/iOS/CloudXRViewer-iOS-Info.plist* 50⟩
Defined in 50. Used in root.

⟨*CloudXRViewer/iOS/LaunchView+iOS.swift* 69⟩
Defined in 69. Used in root.

⟨*CloudXRViewer/iOS/SessionConfigView+iOS.swift* 70⟩
Defined in 70, 71. Used in root.

⟨*CloudXRViewer/iOS/StreamingView+extensions+iOS.swift* 74⟩
Defined in 74. Used in root.

⟨*CloudXRViewer/iOS/StreamingView+iOS.swift* 72⟩
Defined in 72, 73. Used in root.

⟨*CloudXRViewer/visionOS/CloudXRViewer-visionOS-Info.plist* 49⟩
Defined in 49. Used in root.

⟨*CloudXRViewer/visionOS/CloudXRViewer-visionOS.entitlements* 48⟩
Defined in 48. Used in root.

⟨*CloudXRViewer/visionOS/FoveatedImmersiveView.swift* 66⟩
Defined in 66. Used in root.

⟨*CloudXRViewer/visionOS/ImmersiveView.swift* 65⟩
Defined in 65. Used in root.

⟨*CloudXRViewer/visionOS/LaunchView+visionOS.swift* 57⟩
Defined in 57. Used in root.

⟨*CloudXRViewer/visionOS/MainContentView+visionOS.swift* 61⟩
Defined in 61, 62. Used in root.

⟨*CloudXRViewer/visionOS/Placement/Placement+Extensions.swift* 68⟩
Defined in 68. Used in root.

⟨*CloudXRViewer/visionOS/SessionConfigView+visionOS.swift* 58⟩
Defined in 58, 59, 60. Used in root.

⟨*CloudXRViewer/visionOS/UIStyles+visionOS.swift* 67⟩
Defined in 67. Used in root.

⟨*CloudXRViewer/visionOS/WindowUI.swift* 63⟩
Defined in 63, 64. Used in root.

⟨*Config.xcconfig* 47⟩
Defined in 47. Used in root.

⟨*foveated connect flow* 46⟩
Defined in 46. Used in 38.

⟨*MIT permission notice* 1⟩
Defined in 1. Used in 2, 21, 28, 40, 66.

⟨*NVIDIA license header* 2⟩
Defined in 2. Used in 3, 13, 19, 20, 32, 39, 51, 57, 58, 61, 63, 65, 67, 68, 69, 70, 72, 74, 75, 77, 79, 80, 81, 82, 83.